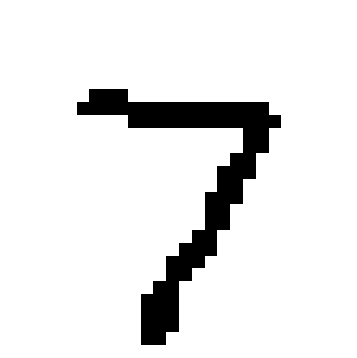
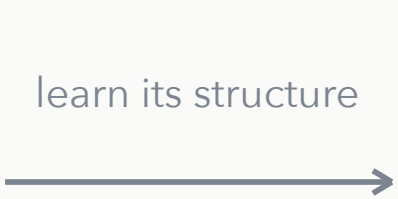
Generative Modelling I

From randomness to data



an observed digit

learn its structure





another plausible example

What must the model learn?

OBSERVED MNIST test digit · original conceptual diagram · Authored course example · Reading

Why generate?

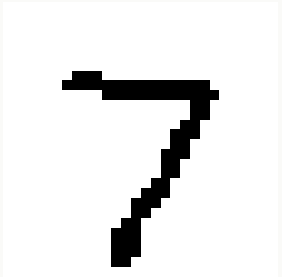
00

What would a useful generator have to learn?

From randomness to data

Recognizing a digit is one task. Drawing one is another.

Given an image



→ 7

Predict a label

Given a collection of examples

learn →



Produce a possible image

What if several answers are reasonable?

Only part is visible



What could be below?

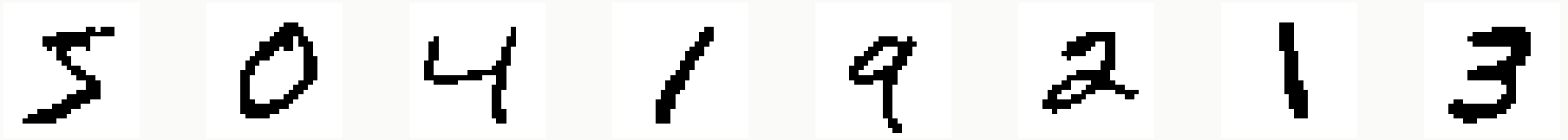
Several plausible completions

A distribution can represent several possible answers.

Application preview: these are authored sketches, not model-generated completions.

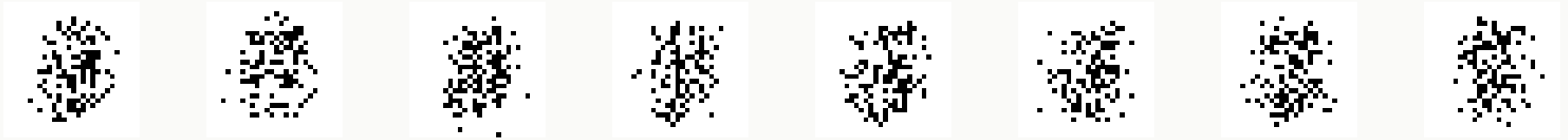
What have these two generators missed?

Replay training images



A sampler that remembers

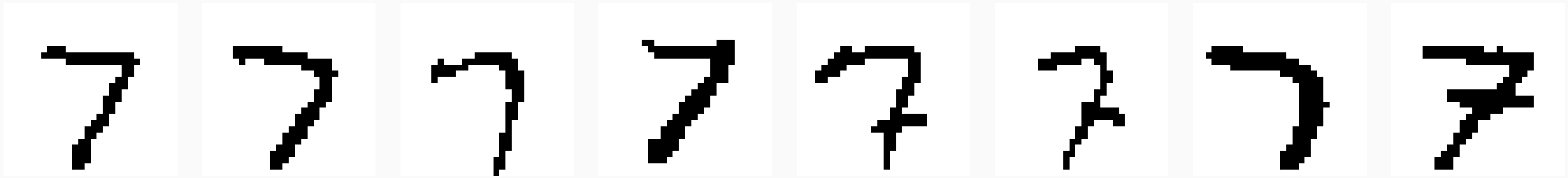
Sample each pixel independently



Matching pixel frequencies does not capture how strokes fit together.

First eight training examples; independent Bernoulli pixels fitted to all 60,000 binary training images, seed 667. No selection.

Which properties of these sevens vary together?

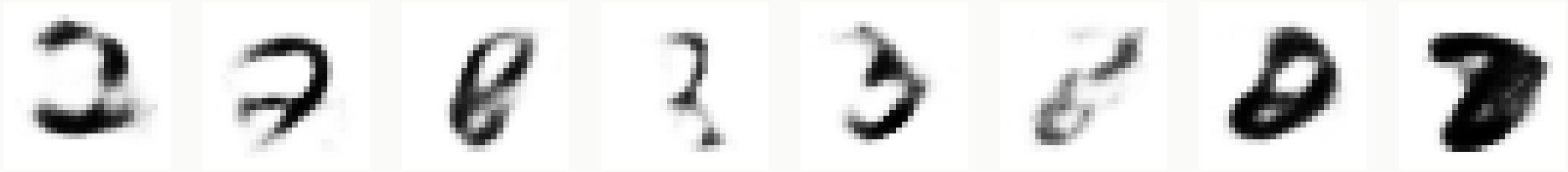


slant thickness height shape

These motivate hidden factors. They are not promised learned axes.

What would count as a good generator?

Actual saved VAE-16 decoder probabilities



Plausibility

Does each example fit?

Coverage

Do we capture variety?

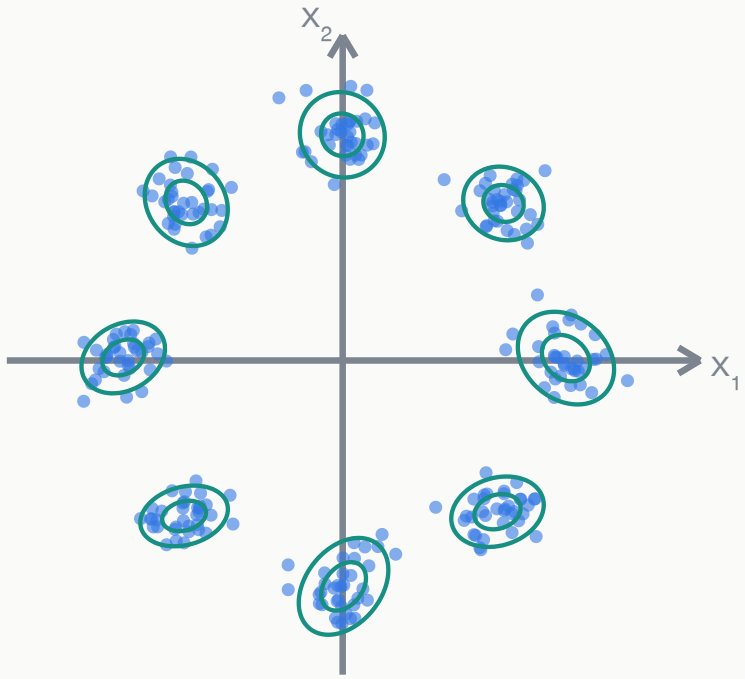
Generalization

Did we learn beyond replay?

A small sample panel cannot answer all three questions.

Saved VAE-16 probabilities; fixed seed, no sample selection.

What does it mean to learn a distribution?



Fitted mixture density

EM learned centres, shapes, weights

Same 280 observations

MEASURED FIT · Gaussian mixture learned by EM on illustrative observations

> Observed points are finite. The distribution describes possible points.

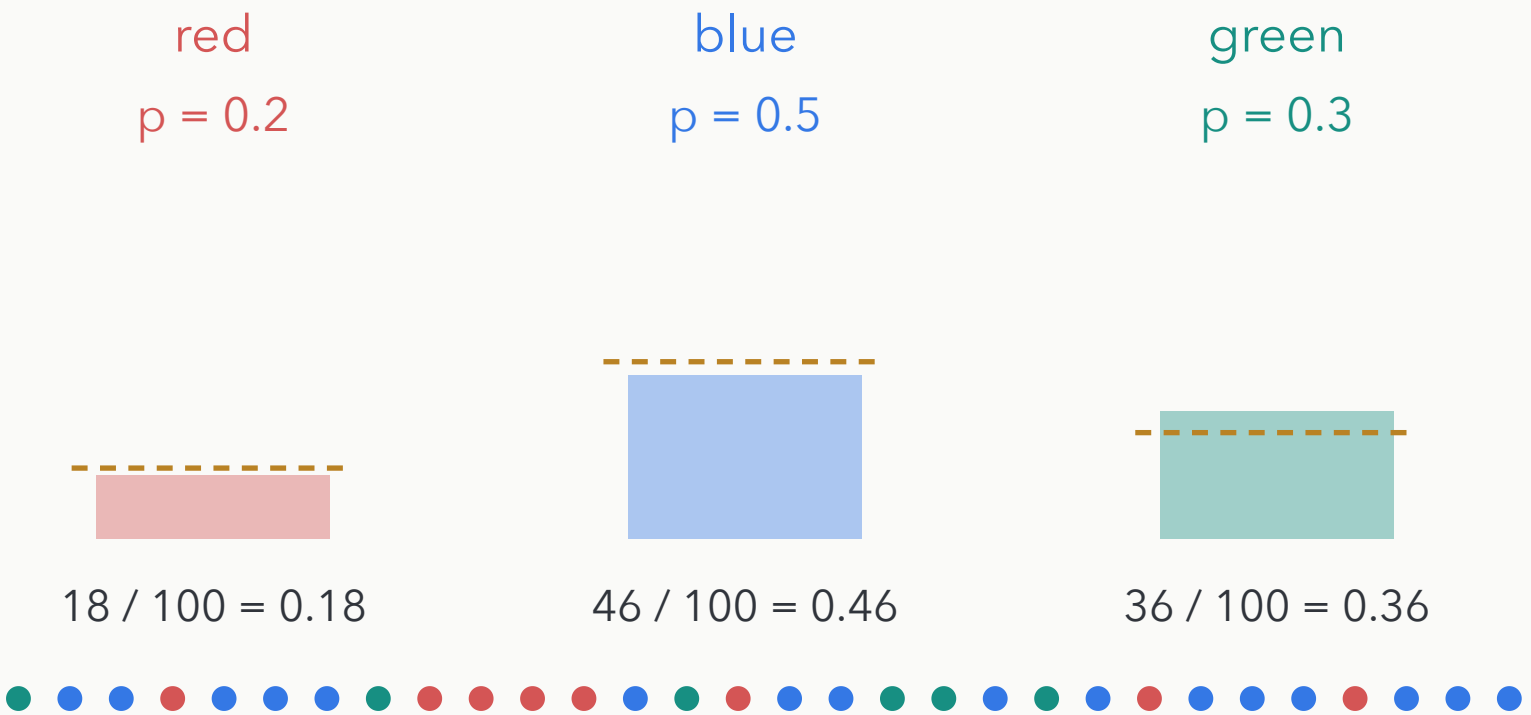
Learning a distribution

01

Where should probability mass go?

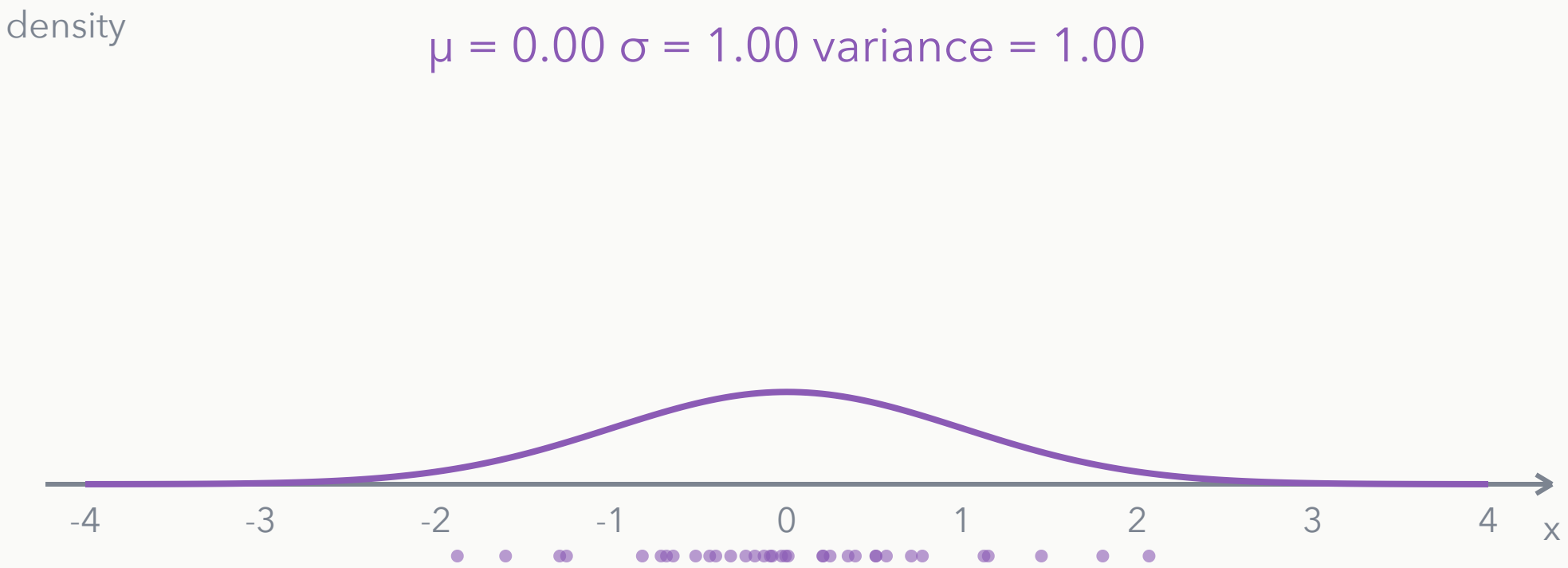
From randomness to data

If we sample 100 times, what might happen?



Compare the observed fractions with 0.2, 0.5 and 0.3.

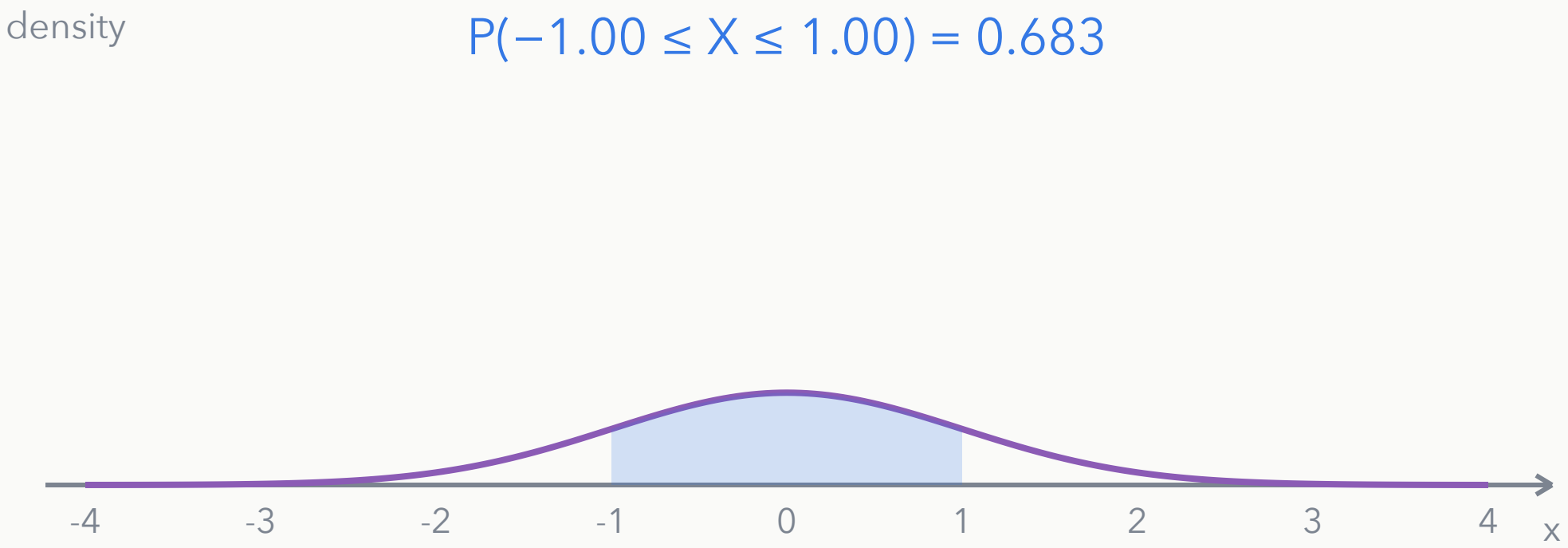
What do μ and σ control?



35 standard-noise draws; points outside the fixed axis range are clipped.

> The mean moves the centre; the standard deviation changes the spread.

Is the height of this curve a probability?

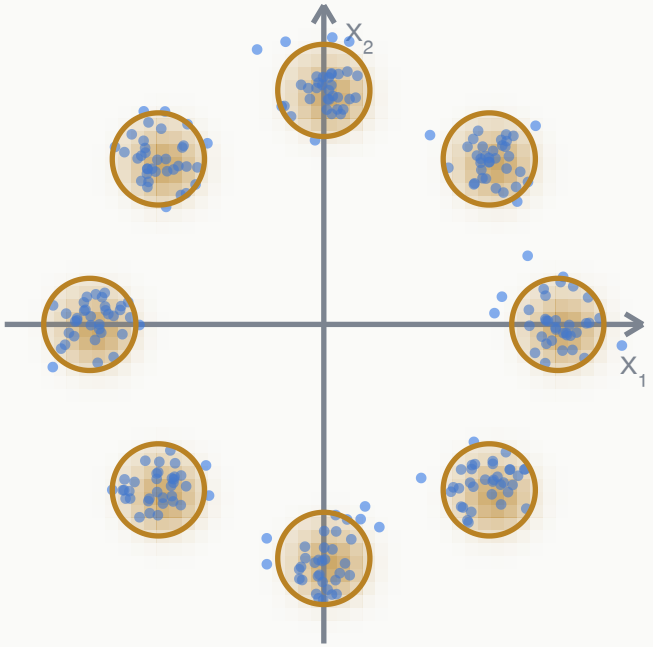


The height is a density. Integrate over the shaded interval.

A point has zero probability. An interval can have positive probability.

Can one Gaussian describe these eight groups?

8 components



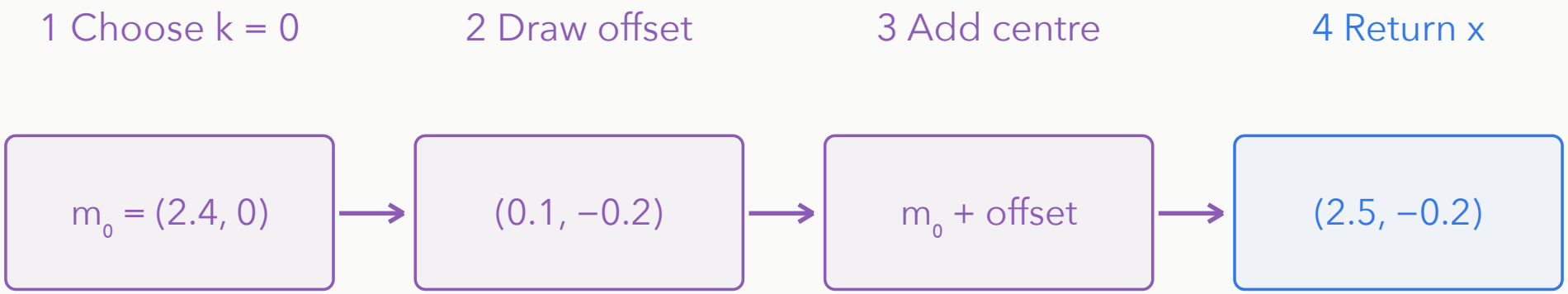
Each weight = 1/8

$$p(x) = \Sigma_k \pi_k N(x; m_k, \sigma^2 I)$$

Blue: balanced teaching sample cloud · amber: known component densities

> The gaps matter as much as the clusters.

Could a simple recipe already generate data?



$$k \sim \text{Categorical}(\pi)\ x \mid k \sim N(m_k, \sigma^2 I)$$

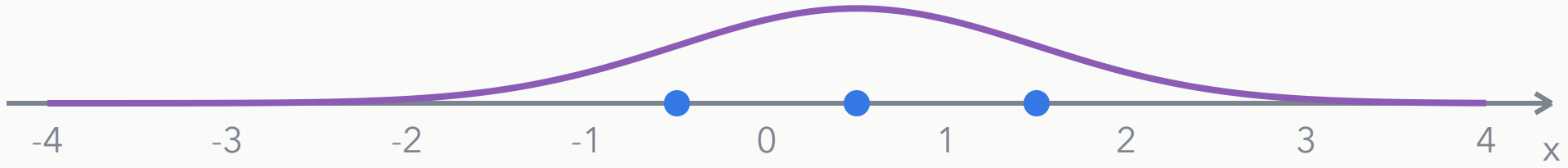
Choose a centre, draw an offset, add them.

Which parameters explain our observations?

density

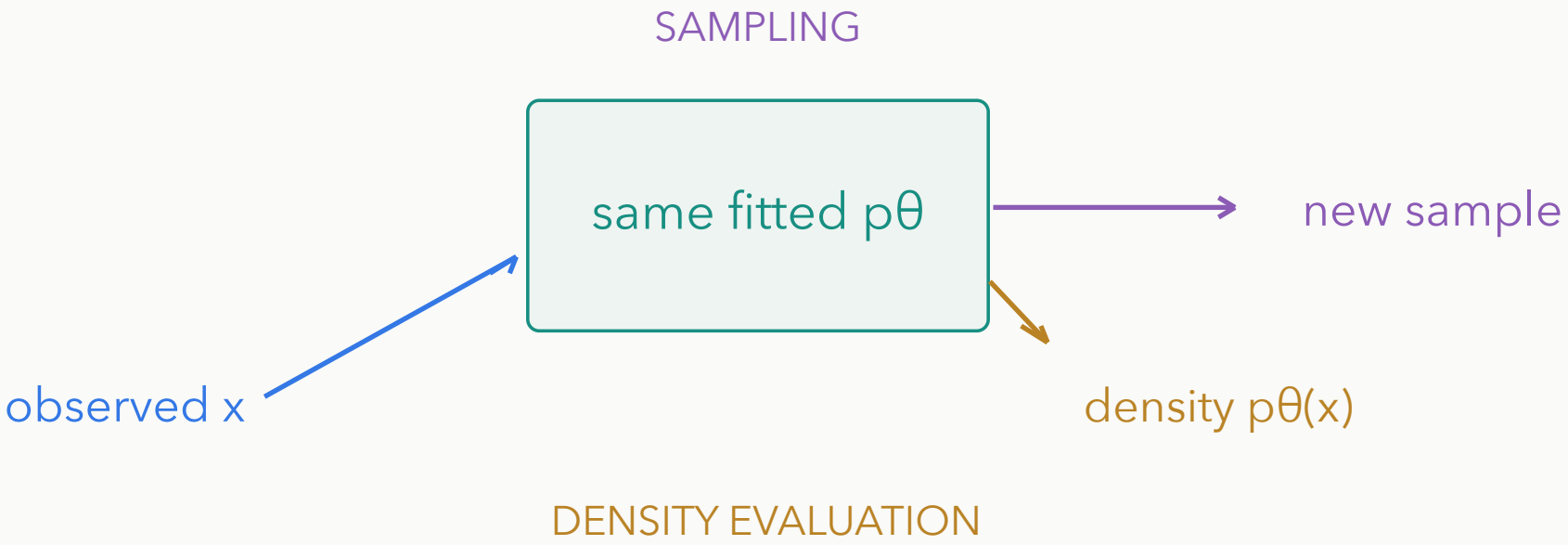
$\mu = 0.50\ \sigma = 1.00$ variance = 1.00

$\Sigma \log p(x_i \mid \mu) = -3.7568$ nats



Maximum likelihood: $\hat{\theta} = \text{argmax}_\theta \Sigma_i \log p_\theta(x_i)$

Can we sample it? Can we evaluate its density?



A sampler and a tractable density are different kinds of access.

We already know one

02

Our language model already generates. How?

From randomness to data

Our decoder-only language model was generative

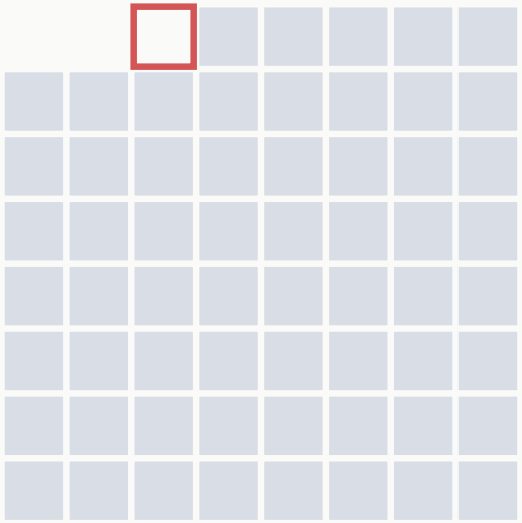
The → The dog → The dog ran

$$p(x_1, \ldots, x_T) = \prod_t p(x_t \mid x_{<t})$$

> Sample a token, append it, repeat.

ILLUSTRATIVE token sequence · exact autoregressive factorization · Authored course example

Could we do the same for image pixels?



Predict pixel 3

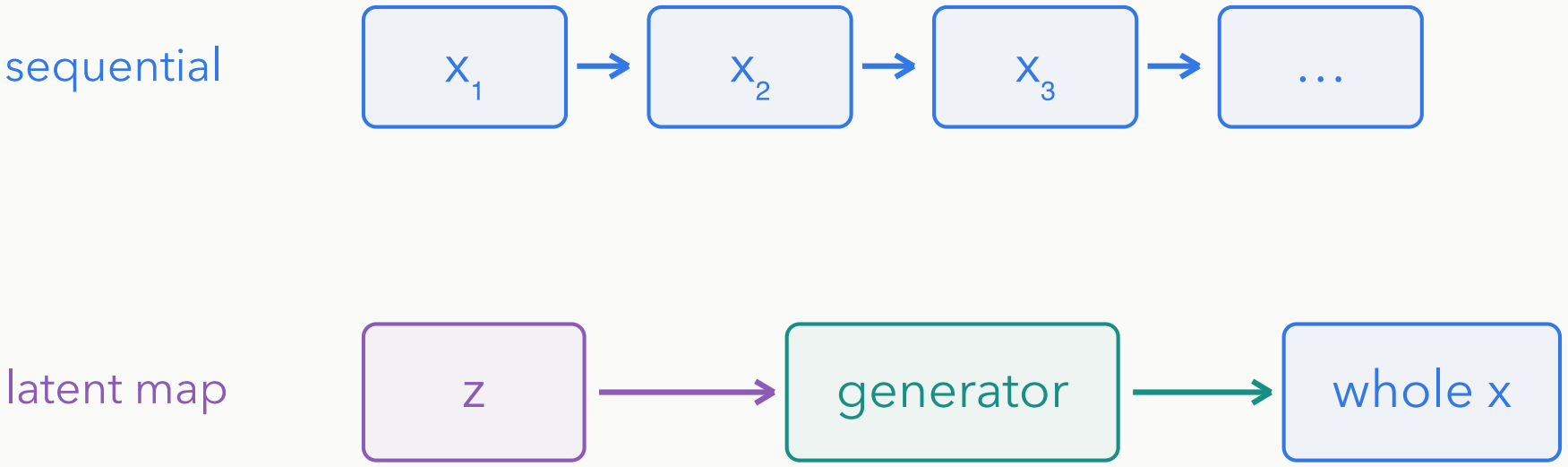
Condition on the earlier pixels.

784 pixel decisions for MNIST

$$p(x) = \prod_{j=1}^{784} p(x_j \mid x_{<j})$$

> Choose an order, then predict each part from earlier parts.

Must every part wait for the previous part?



Could one learned transformation turn randomness into a whole object?

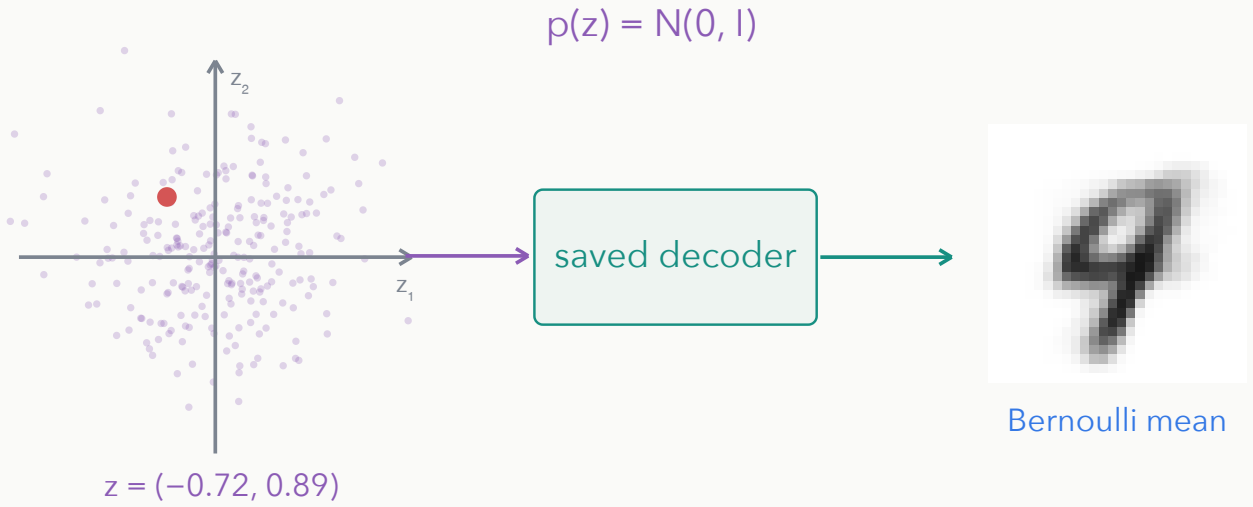
From randomness to data

03

Why can reconstruction succeed while sampling fails?

From randomness to data

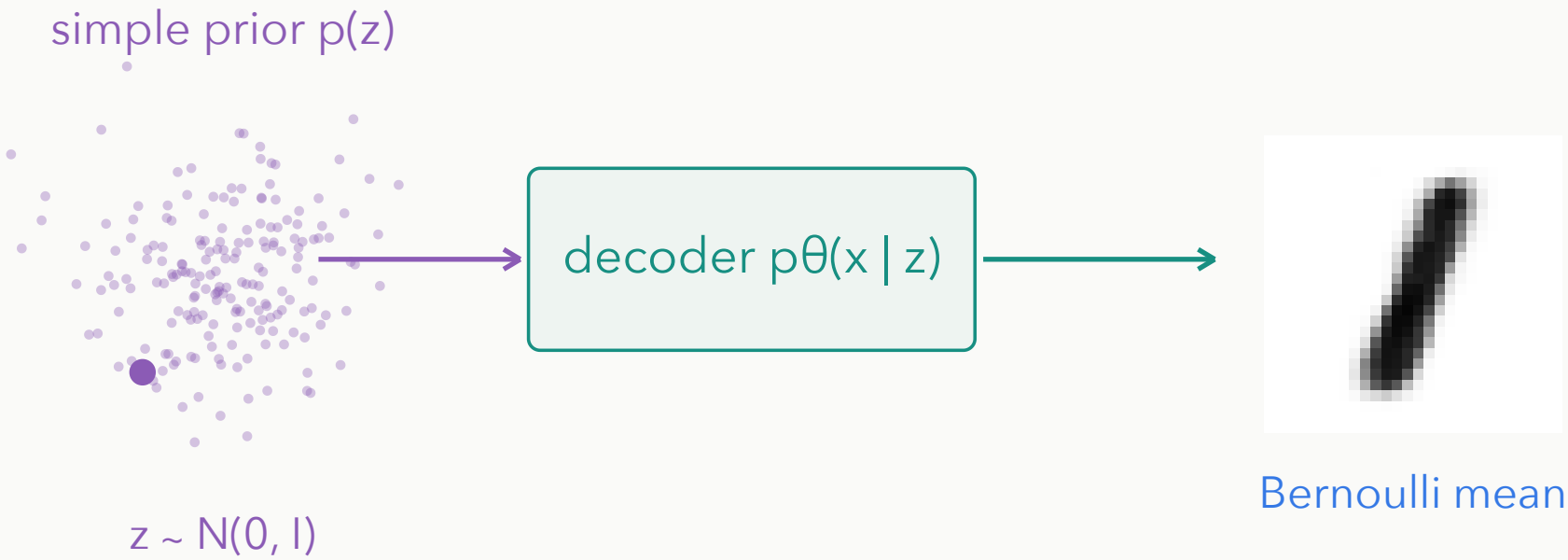
Where will our randomness come from?



The standard normal is our chosen prior; generative models can use other priors.

> The prior chooses z. The saved decoder determines the image.

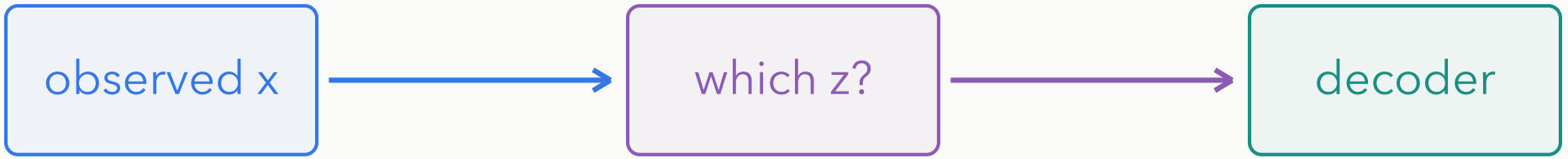
What should a neural generator do with z?



The decoder returns probabilities. A pixel draw supplies the binary image.

> A deterministic generator returns x; our decoder returns Bernoulli probabilities.

During training we see x. Which z explains it?



The hidden cause was not supplied with the training example.

Could an encoder find a useful code?



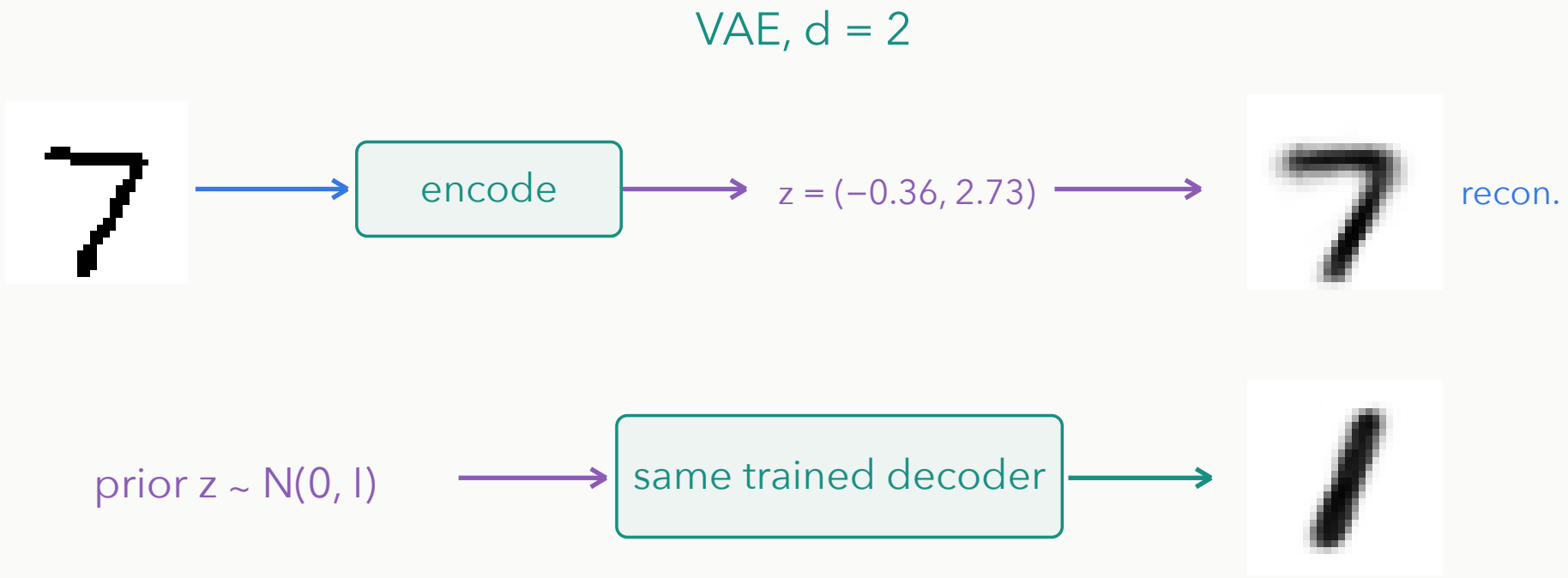


Top: observed digits · bottom: decoder means

> An ordinary autoencoder learns to reconstruct its input.

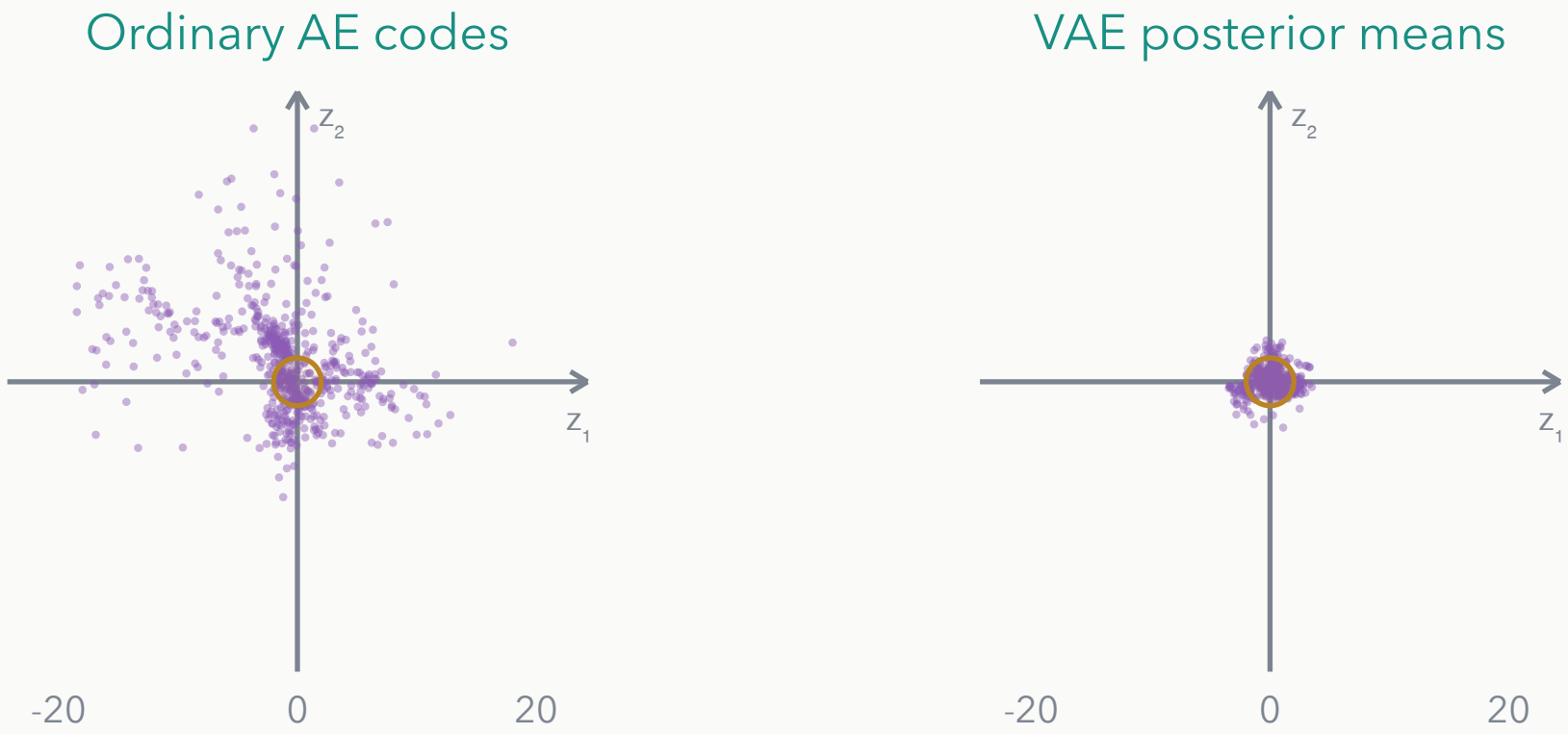
MEASURED · AE-2 reconstruction means on first held-out examples · Authored course example

Reconstruct a digit. Then try a prior draw.



First test digit and first fixed-seed prior vector; no selection or retouching.

Will a random Gaussian code decode well?



Same 600 test inputs · amber radius 2 under N(0, I) · identical axis scales

> Reconstruction alone does not make AE codes match a chosen prior.

MEASURED latent means · amber 2σ prior circle; same coordinate scale for AE and VAE · Authored course example · Reading

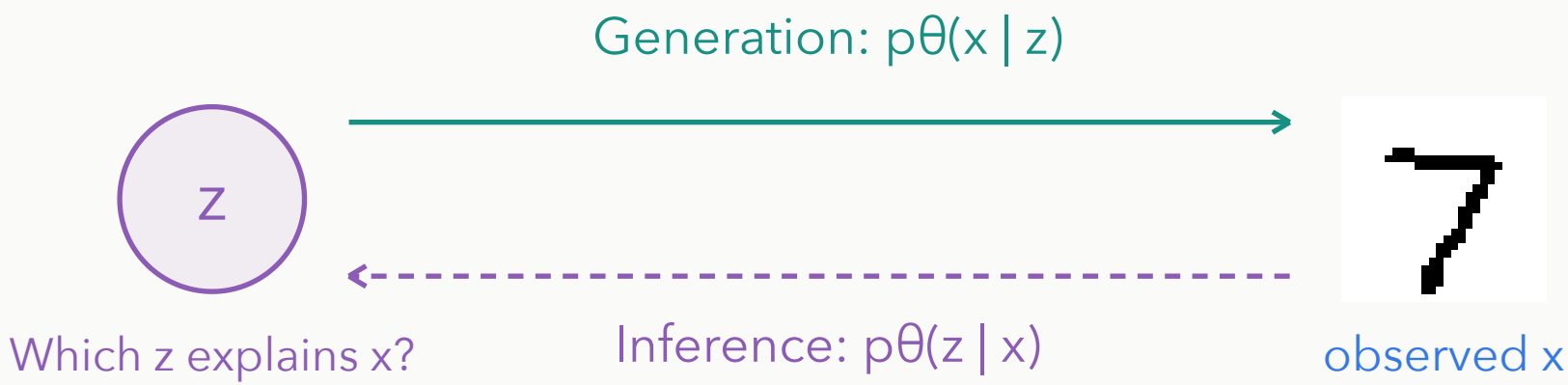
Inference, KL and the bound

04

Which latent values explain this image?

From randomness to data

The same model, viewed in two directions



The reverse question needs a distribution over explanations.

An inference network is not an inverse decoder.

How probable is this observation?

A tiny model with three latent choices

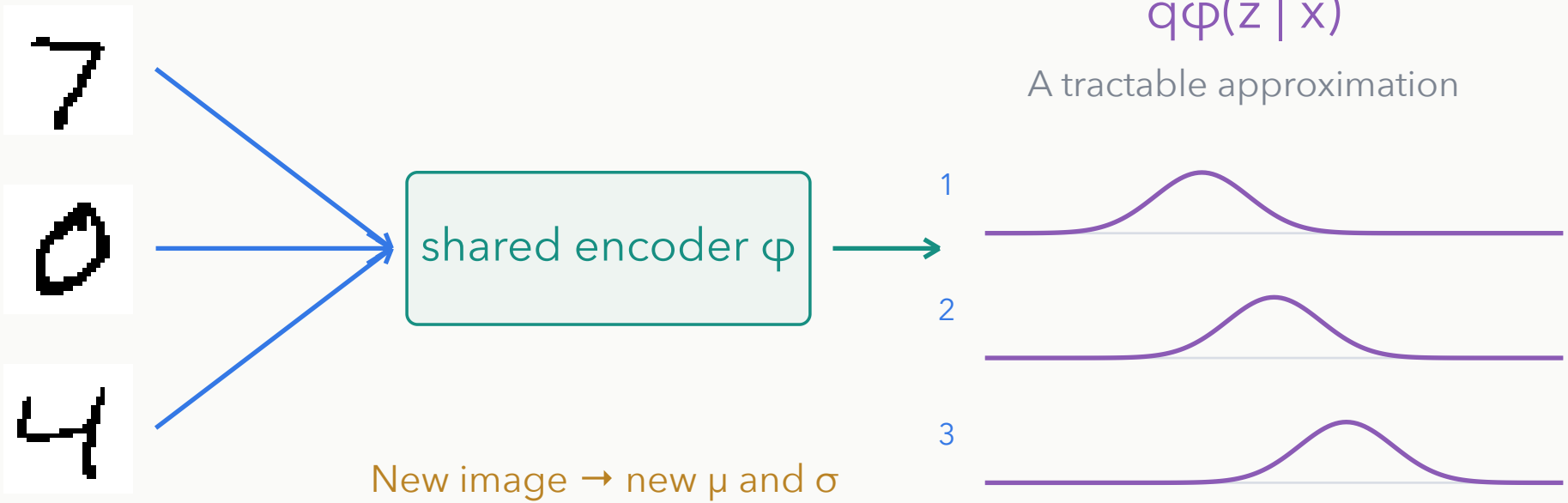
	z = A	z = B	z = C
prior p(z)	0.20	0.50	0.30
likelihood p(x | z)	0.10	0.80	0.20
joint contribution	0.02	0.40	0.06

$$p(x) = 0.02 + 0.40 + 0.06 = 0.48$$

Continuous z: replace the sum with $\int p_\theta(x \mid z)\, p(z)\, dz$.

Exact finite example; neural latent models generally need approximate inference.

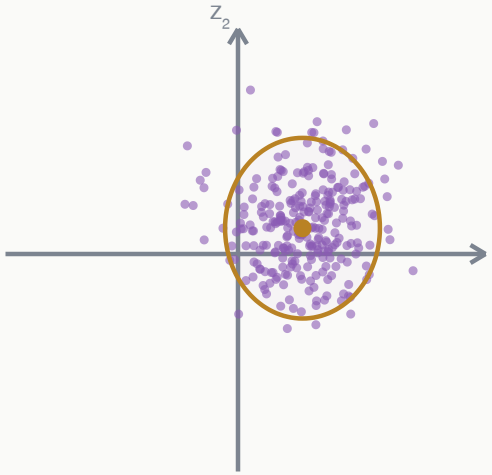
Let one encoder learn to propose explanations



Amortized inference: learn one function instead of fitting each image from scratch.

Approximate posterior qφ(z | x), not the true posterior.

What if the encoder predicts a distribution?



One input → one distribution

Mean sets the centre.

Standard deviations set spread.

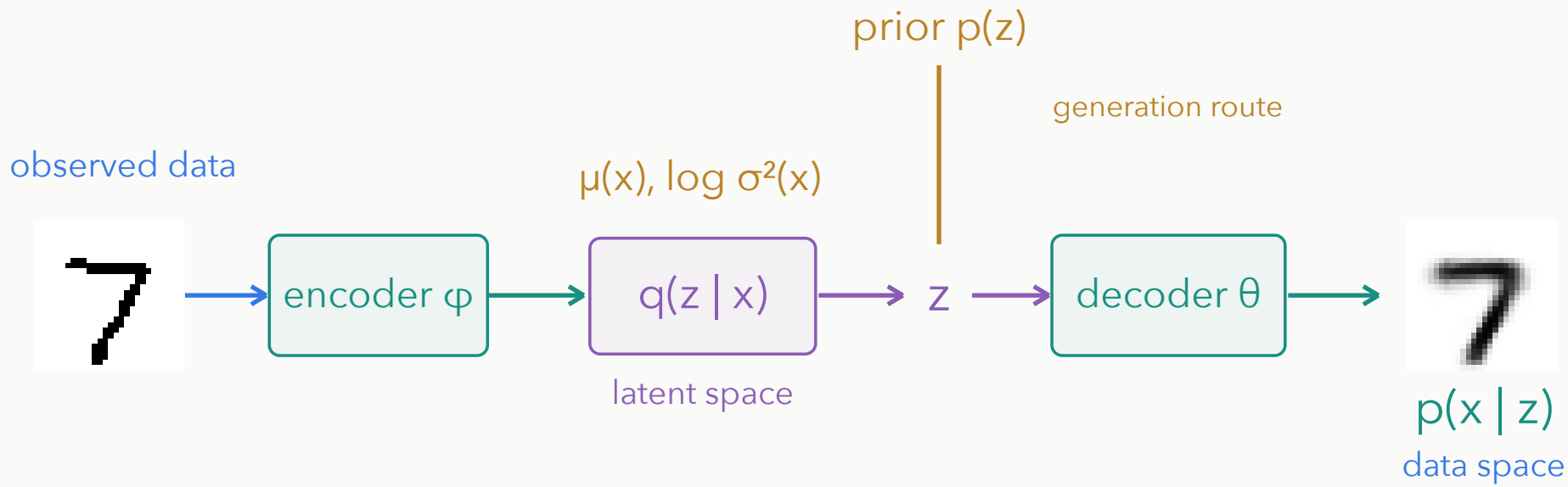
The ellipse is a 2σ contour.

μ = (μ₁, 0.4), σ = (σ₁, 0.7) · illustrative parameters

$$\mu_\phi(x),\ \log \sigma^2_\phi(x) \in \mathbb{R}^d, \qquad q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x))\big)$$

ILLUSTRATIVE posterior ellipse · d=2 or d=16 in measured runs · Source

Which distribution are we talking about?



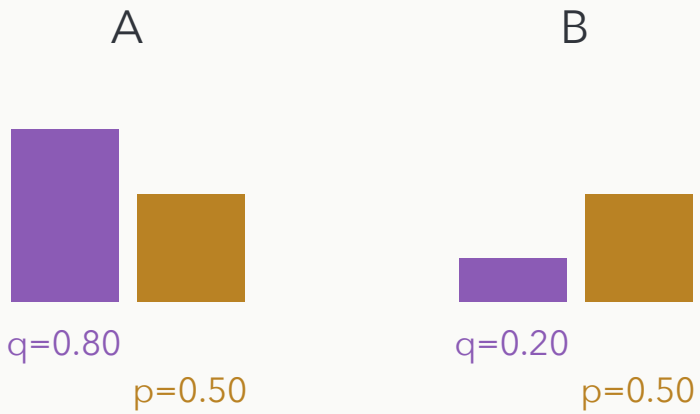
Inference uses q because the exact posterior can be intractable.

> p and q describe latent variables; the decoder describes observations.

How expensive is the wrong probability model?

Outcomes are drawn from q

Score them using p



extra log loss at each outcome

A: $\log(q/p) = 0.470$

B: $\log(q/p) = -0.916$

$$0.80 \times (0.470) + 0.20 \times (-0.916)$$

$$KL(q \parallel p) = 0.193 \text{ nats}$$

Natural logarithms → nats. Individual contributions can be negative; total KL ≥ 0.

Why does the direction of KL matter?

$$q = (0.8, 0.2)\ p = (0.5, 0.5)$$

Draw from q

$$0.8 \log(0.8/0.5) + 0.2 \log(0.2/0.5)$$

0.193 nats

Draw from p

$$0.5 \log(0.5/0.8) + 0.5 \log(0.5/0.2)$$

0.223 nats

The first distribution determines the averaging weights. KL is not symmetric.

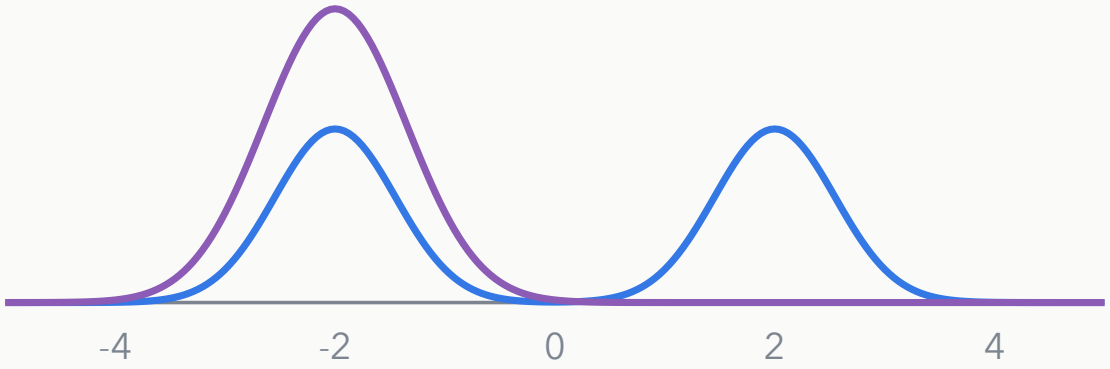
If p assigns zero probability where q has positive mass, $KL(q \Vert p)$ is infinite.

Cover both groups, or concentrate on one?

One Gaussian approximation to two separated groups

target p

approximation q



Average under p

$KL(p \parallel q) = 8.800$

Average under q

$KL(q \parallel p) = 0.721$

Concentrating on one group leaves the other group poorly covered.

Two illustrative candidate fits, not optimized solutions. Both scores use numerical quadrature.

Can we average over a distribution we can sample?

Same fixed model. Choose q = (0.4, 0.3, 0.3).

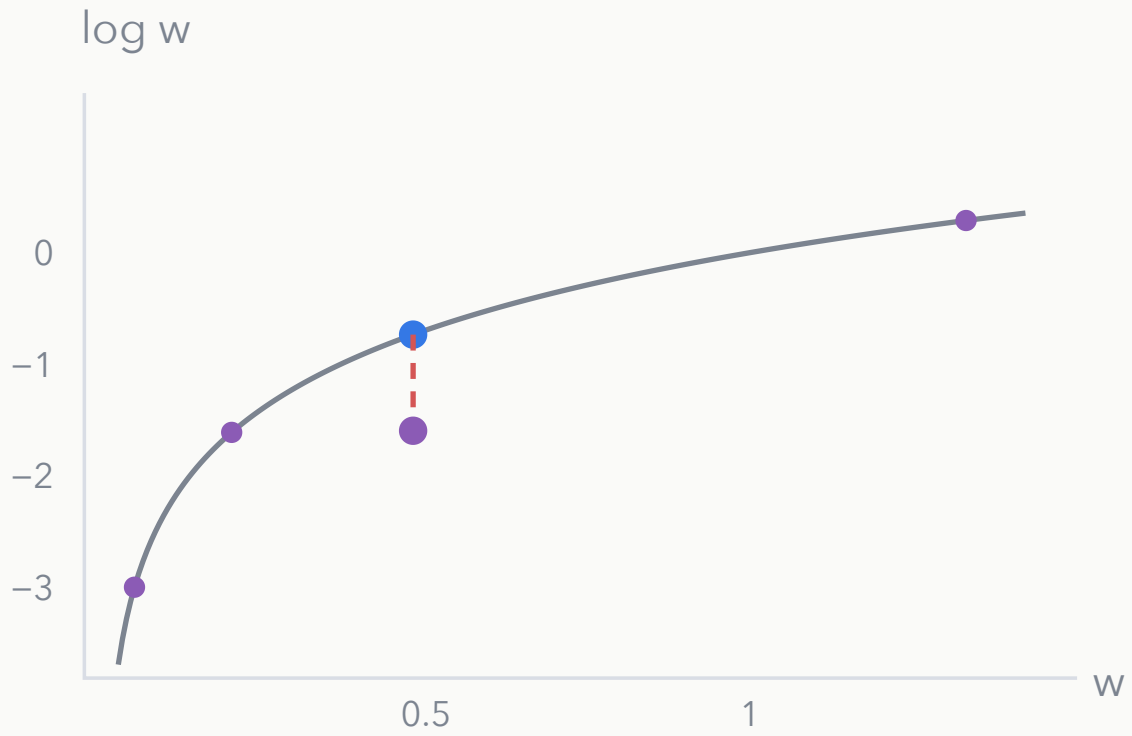
	A	B	C
joint p(x,z)	0.02	0.40	0.06
proposal q(z | x)	0.40	0.30	0.30
weight w = joint / q	0.050	1.333	0.200

$$E_q[w] = 0.4 \times 0.05 + 0.3 \times 1.333\ldots + 0.3 \times 0.2$$

$$E_q[w] = 0.48 = p(x)$$

q must be positive wherever p(x,z) contributes to the evidence.

What changes when we move the log inside?



Log after averaging

$\log E_q[w] = \log 0.48$

$= -0.734$

Average after taking logs

$E_q[\log w] = -1.595$

$\text{ELBO} \leq \log p(x)$

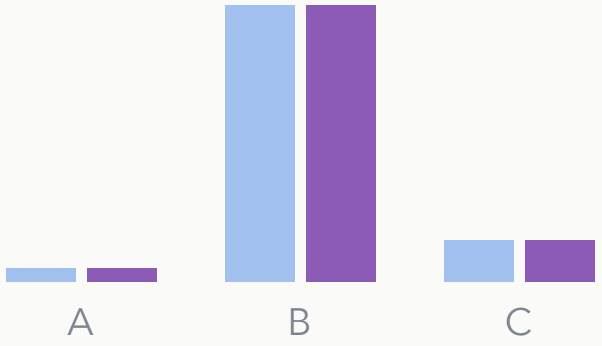
Jensen's inequality

ELBO: evidence lower bound. Small dots: log weights; large purple dot: their q-weighted average.

How can we improve a bound on the evidence?

Same fixed generative model

p(x) = 0.48



blue: true posterior purple: q

log p(x) = −0.734

q equals the true posterior

The gap is zero.

ELBO = −0.734

$$\log p(x) = \mathrm{ELBO} + \mathrm{KL}(q \,\|\, \text{true posterior})$$

Exact toy calculation. θ is fixed here; changing θ changes the evidence too.

Two KL terms. Two different comparisons.

The gap: a comparison we cannot usually compute

$$\log p_\theta(x) - \mathrm{ELBO} = \mathrm{KL}(q_\varphi(z \mid x) \,\|\, p_\theta(z \mid x))$$

true posterior

The objective: terms we can estimate or compute

$$\mathrm{ELBO} = \mathbb{E}_q[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\varphi(z \mid x) \,\|\, p(z))$$

reconstruction likelihood

prior

The prior $p(z)$ and the posterior $p_\theta(z \mid x)$ are different distributions.

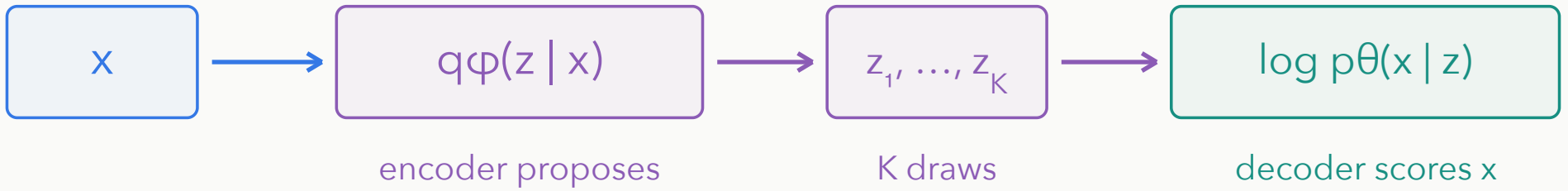
Why does the VAE loss have two parts?

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_q[\log p_\theta(x \mid z)] + D_{\text{KL}}(q_\phi(z \mid x) \| p(z))$$

The complete negative ELBO, with β = 1.

Which parts can we compute and differentiate?

A computable objective for each observed x



ELBO estimate = average log likelihood − KL(qφ(z | x) ‖ p(z))

Minimise the negative bound over examples; update both φ and θ.

Gaussian prior KL is exact here. The expectation uses samples.

K = 1 is common. β = 1 gives the ELBO; changing β changes the objective.

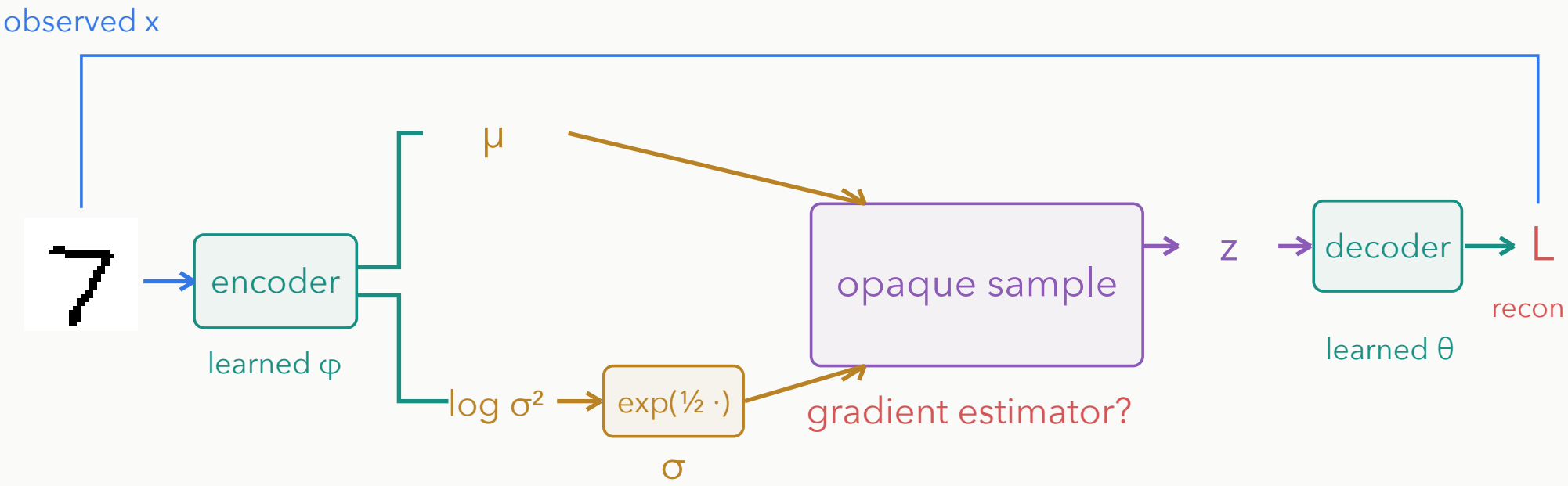
Training through randomness

05

How does a gradient pass through a random draw?

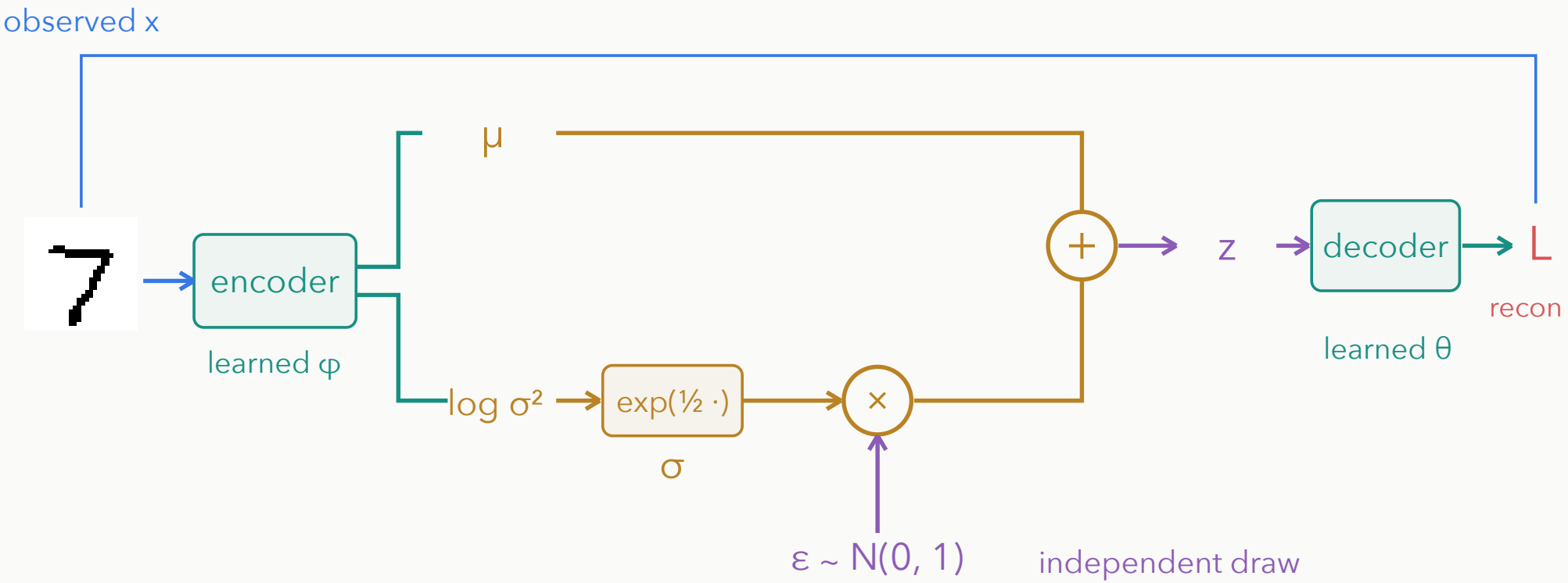
From randomness to data

Where should the gradient go?



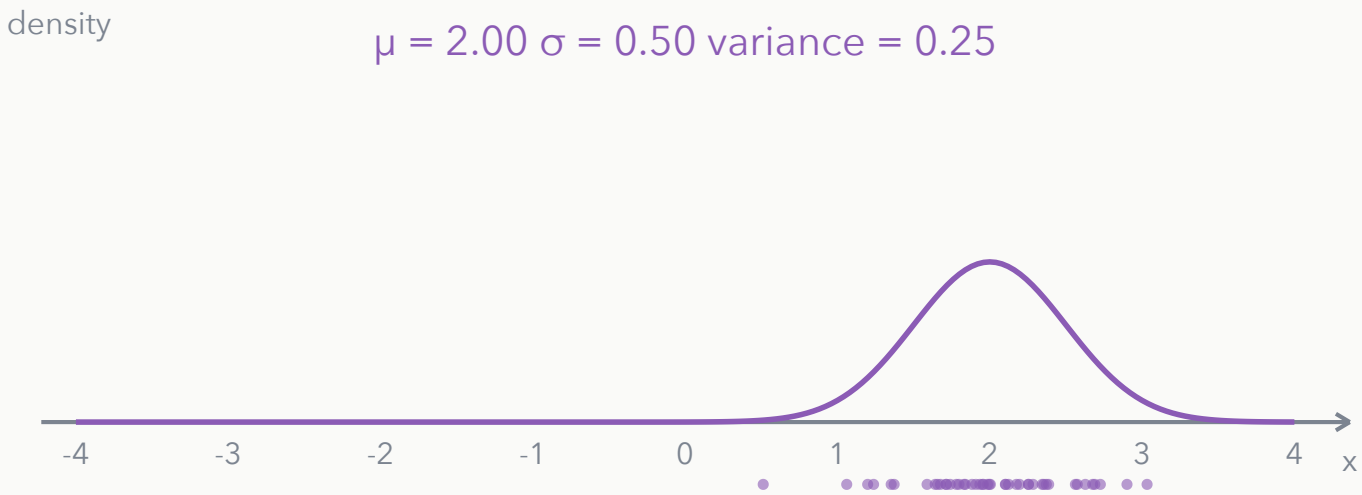
An opaque stochastic operation leaves the gradient estimator unspecified.

Put the randomness in a separate input



With ε fixed, $z = \mu + \sigma\varepsilon$ is a differentiable computation.

Why does scale-and-shift give the right distribution?

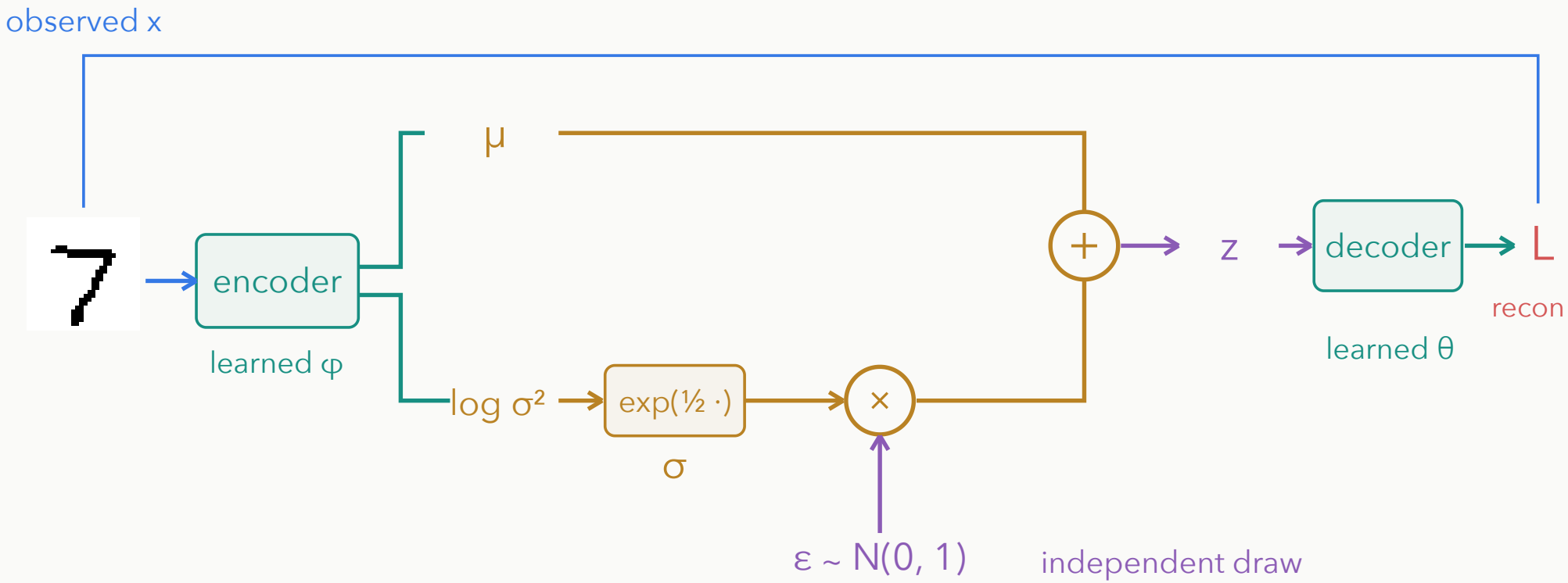


μ + σε: add 2; mean 2, standard deviation 0.5.

> Use the same ε draws: scale first, then add μ.

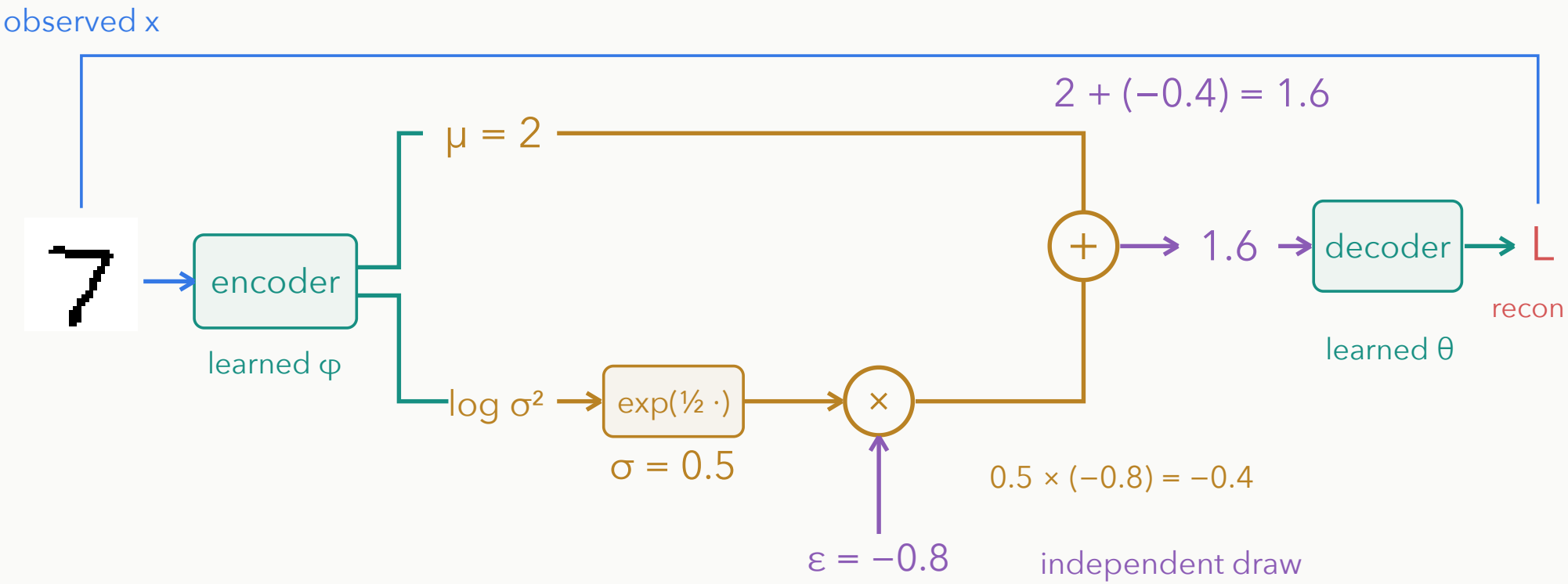
ANALYTICAL / EXACT identity · same illustrative standard-noise draws · Source

One graph, from image to reconstruction loss



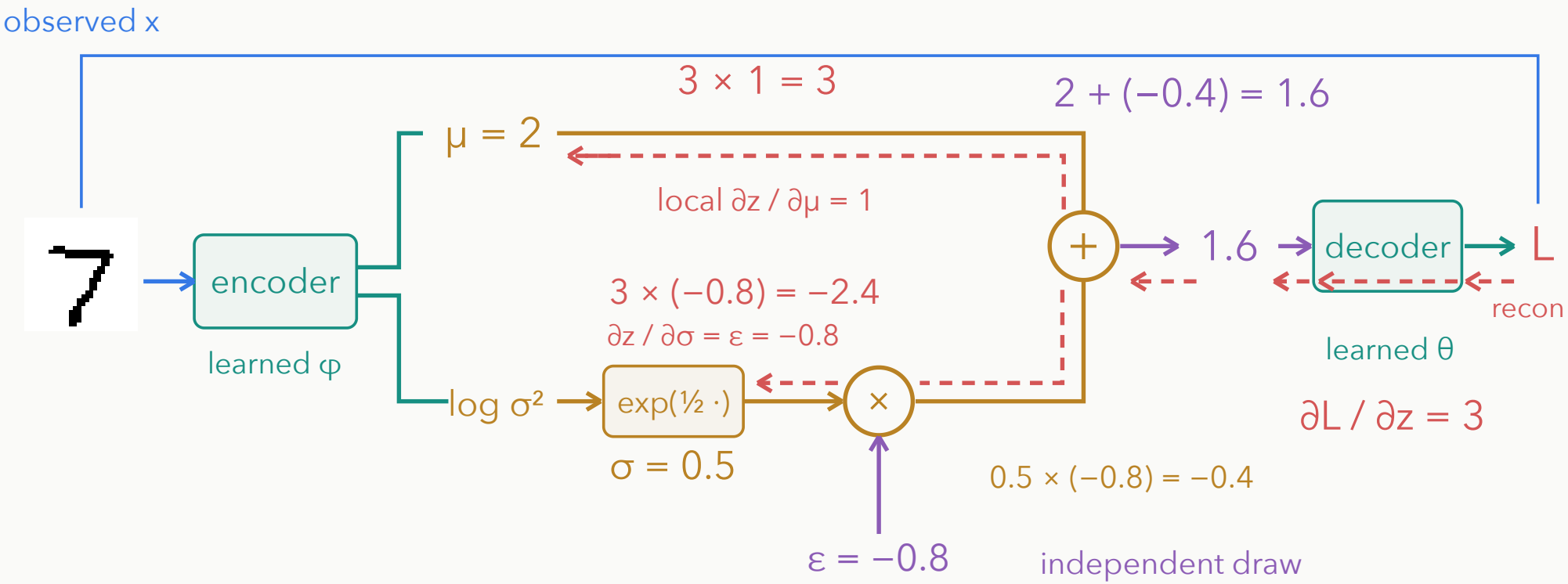
With ε fixed, z = μ + σε is a differentiable computation.

Follow one draw through the graph



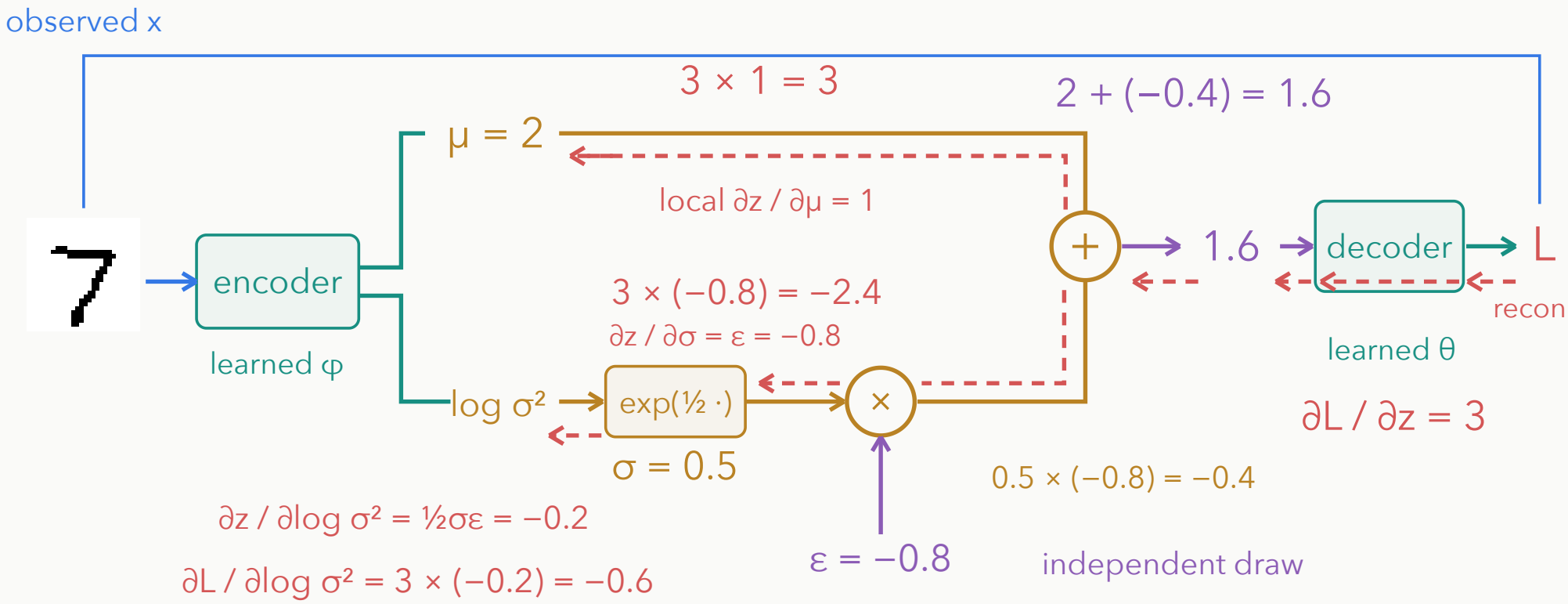
Add the mean. This is the latent value sent to the decoder.

Now follow the same graph backwards



The same chain rule gives the gradient at σ.

Why does the encoder return log-variance?



These are reconstruction-path gradients. The full VAE also adds KL gradients.

Four lines implement the same graph

```
mu, logvar = encoder(x)
std = exp(0.5 * logvar)
eps = randn_like(std)
z = mu + std * eps
```

$$\sigma = \exp(\tfrac{1}{2}\log\sigma^2)$$

Differentiate with the sampled eps held fixed.

Which draw keeps the gradient path?

Same distribution. Different autograd behavior.

`Normal(mu, std).sample()`	no pathwise autograd connection
`mu + std * eps`	explicit differentiable path
`Normal(mu, std).rsample()`	the reparameterized interface

For $z = \mu + \sigma\varepsilon$: $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$. Keep ε fixed in this backward pass.

PyTorch behavior verified locally. Other stochastic-gradient estimators also exist.

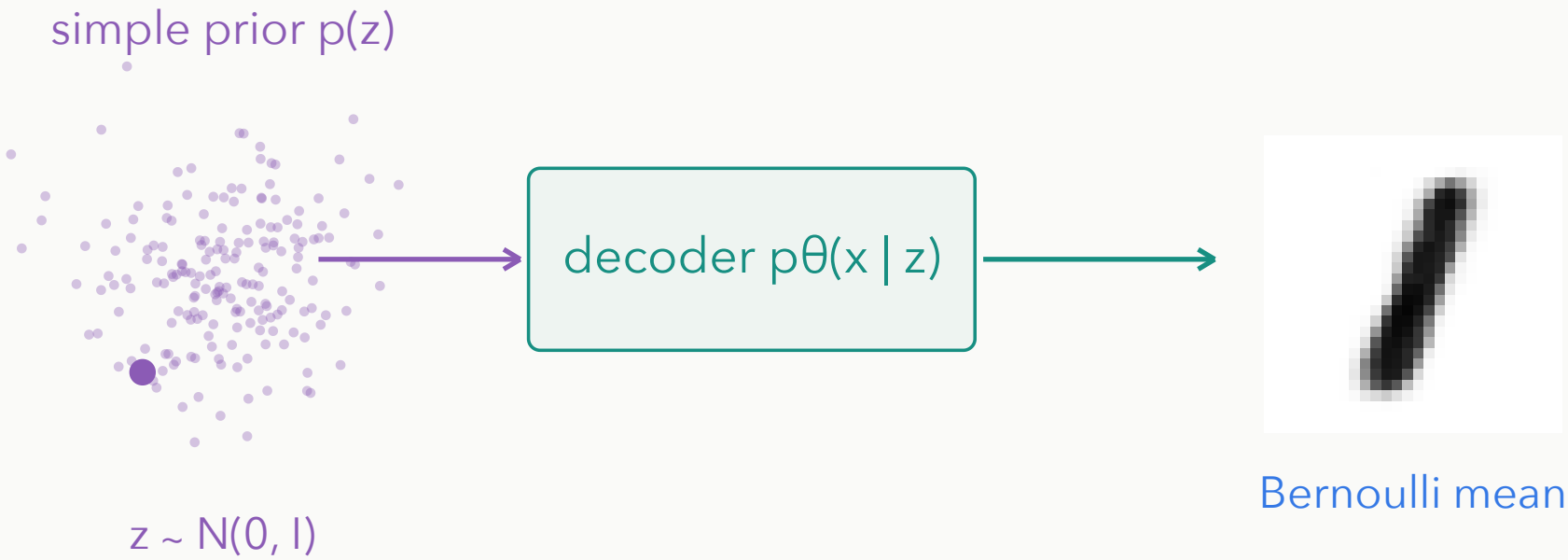
What does the trained model learn?

06

How do reconstruction and generation change together?

From randomness to data

Why not optimize reconstruction alone?



Use both: explain x and compare q(z | x) with p(z).

> Reconstruct each image while keeping its posterior compatible with the prior.

What does “reconstruction loss” actually mean?

Gaussian output	Fixed variance → squared-error-like loss
Bernoulli output	Binary observations → binary cross entropy

$$-\log p_\theta(x \mid z) \text{ depends on the observation model}$$

> Our experiment uses binary MNIST and Bernoulli likelihood.

ANALYTICAL / EXACT · fixed threshold x > 127 applied to uint8 pixels · Source

What probability did we assign to these four pixels?



x = 1

p = 0.8

p = 0.8

−log = 0.223



x = 0

p = 0.3

1 − p = 0.7

−log = 0.357



x = 1

p = 0.6

p = 0.6

−log = 0.511



x = 0

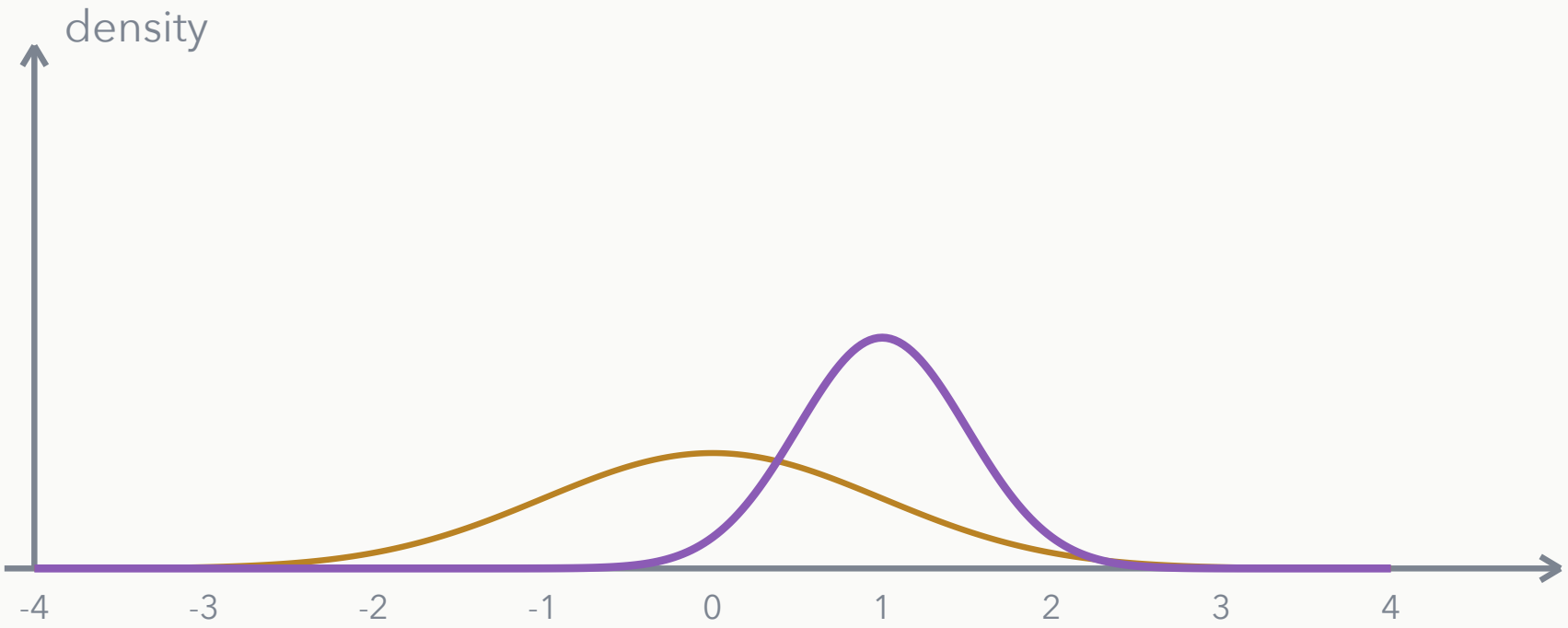
p = 0.1

1 − p = 0.9

−log = 0.105

Sum = 1.196 nats · mean over 4 pixels = 0.299

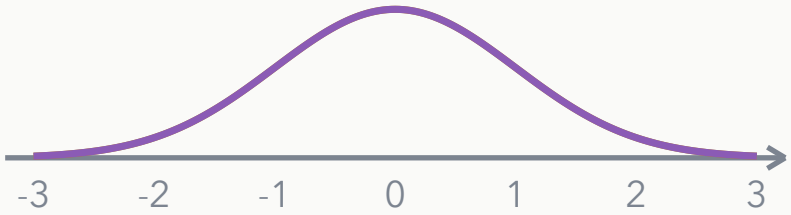
How far is this posterior from the prior?



KL(q ‖ p) = 0.818 nats

ANALYTICAL / EXACT · purple q(z|x), amber p(z)=N(0,1) · Source

Can we calculate one KL by hand?



amber prior · purple posterior

$$\tfrac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

$$\mu^2 = 0$$

$$\sigma^2 = 1$$

$$\log \sigma^2 = 0$$

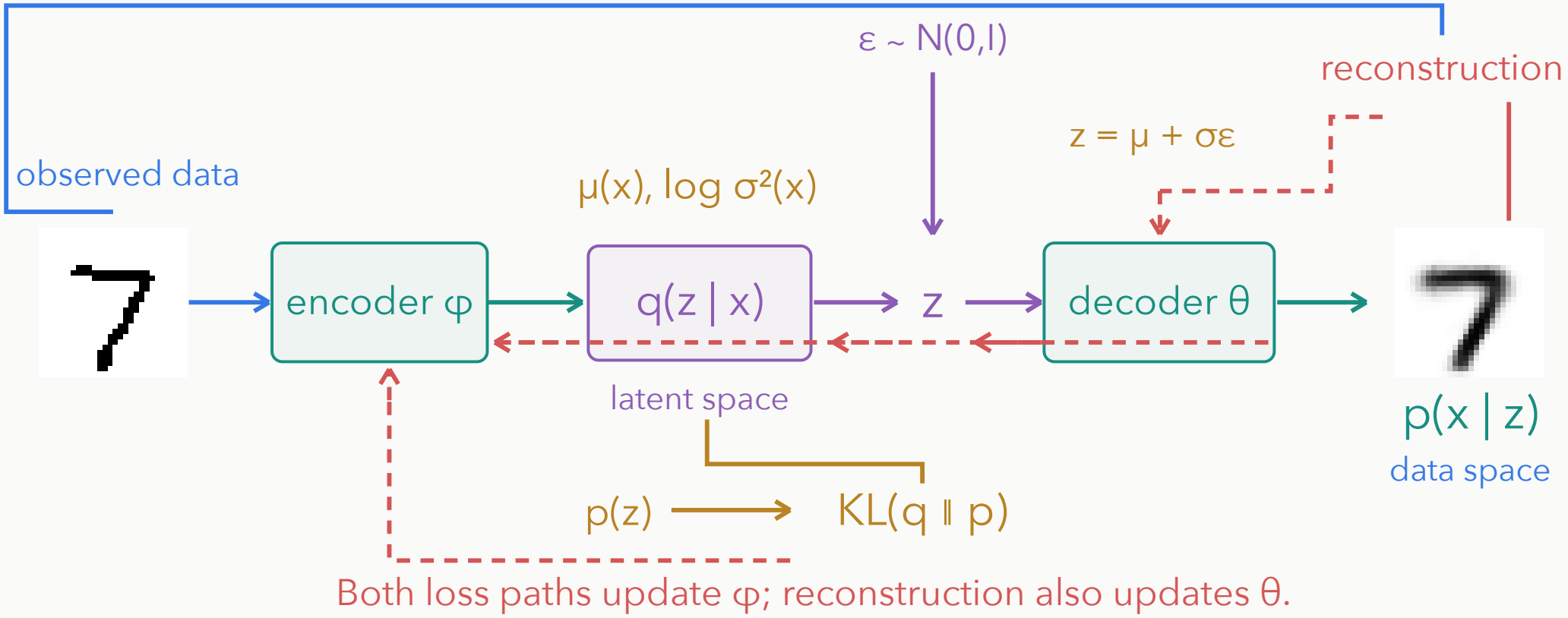
$$0 + 1 - 0 - 1 = 0$$

$$KL = 0$$

This minimizes the KL term alone; reconstruction still matters in the VAE.

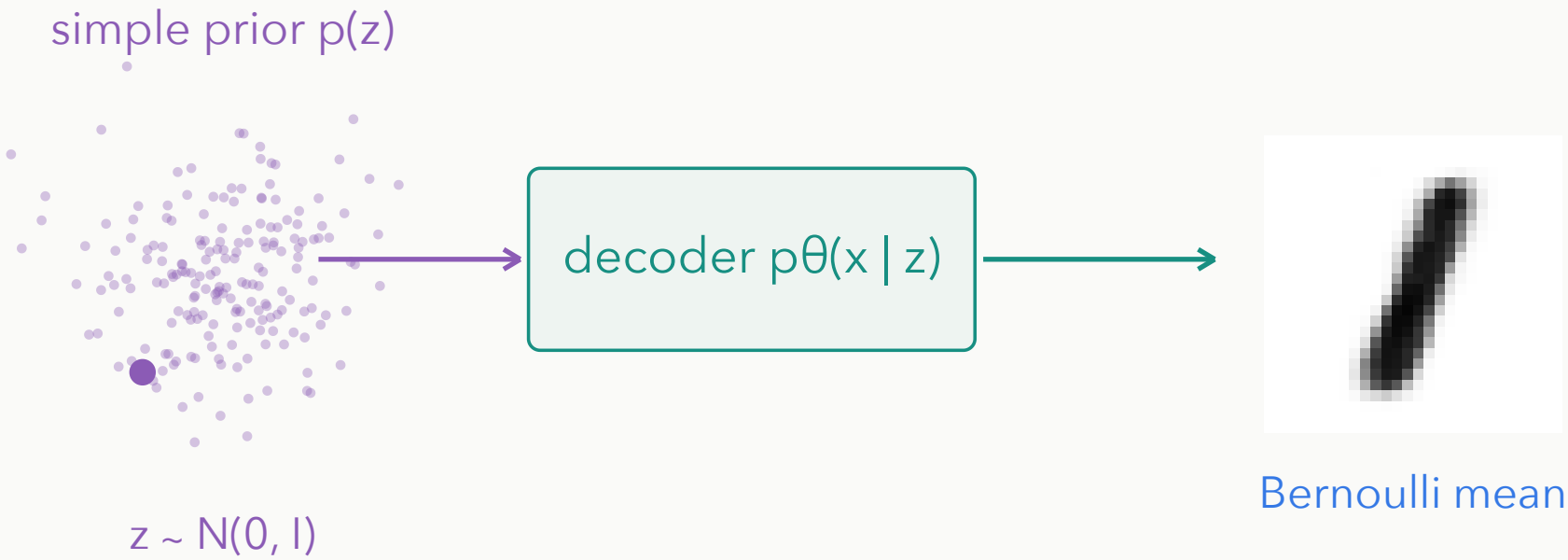
ANALYTICAL / EXACT · natural logarithm; rounded to three decimals · Source

Now we can assemble the VAE



Reconstruction and KL both send gradients to the encoder.

What stays when we only want a new sample?



The decoder returns probabilities. A pixel draw supplies the binary image.

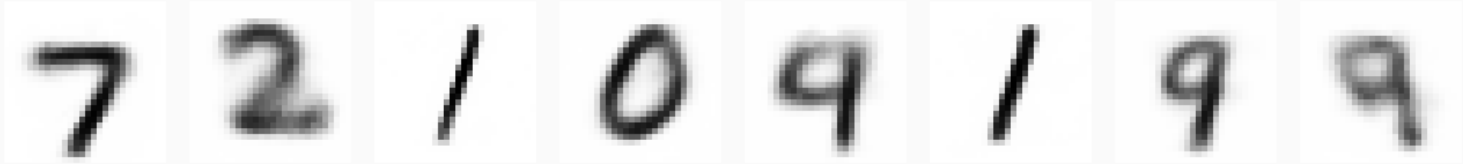
To generate, draw z from the prior and keep the decoder.

How well do the two models reconstruct the same digits?

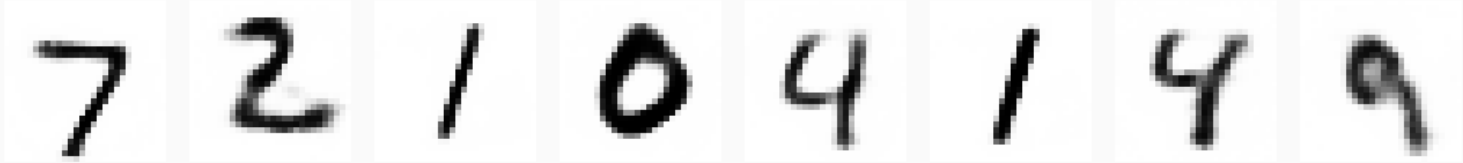
Same eight held-out inputs



VAE-2 reconstruction means



VAE-16 reconstruction means



What happens when we draw from the prior?

VAE-2: sampled binary pixels

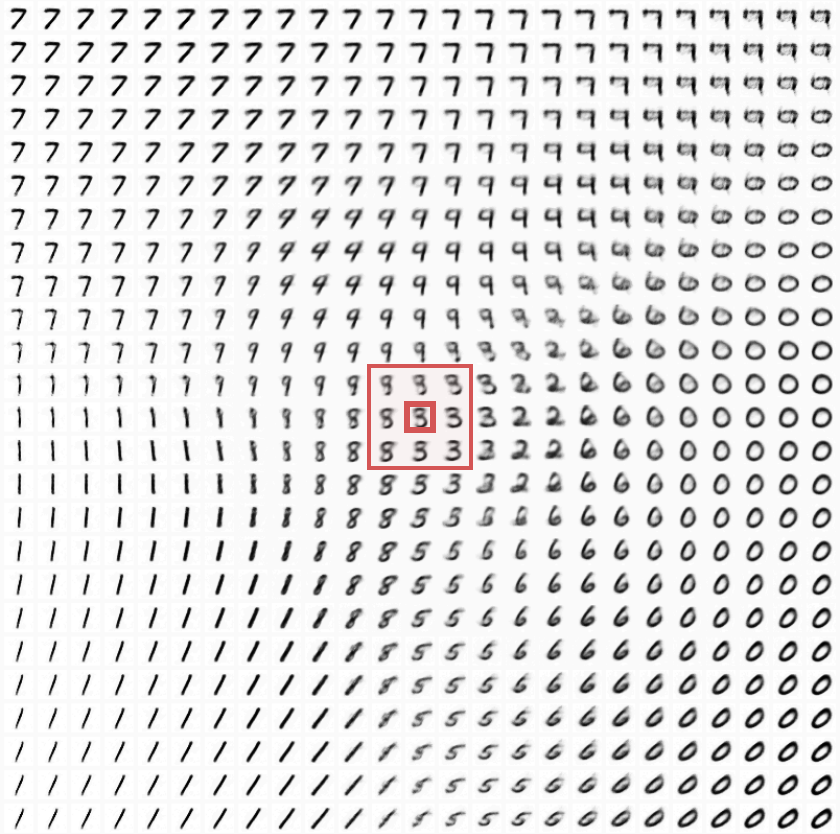


VAE-16: sampled binary pixels



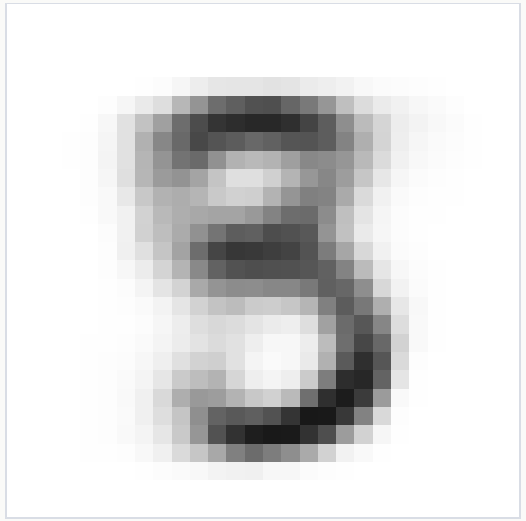
MEASURED · saved checkpoints; prior seed 667; Bernoulli seed 668; no selection · Source · Reading

What lives at this point in latent space?



z_1: −3 → 3 · z_2: 3 → −3

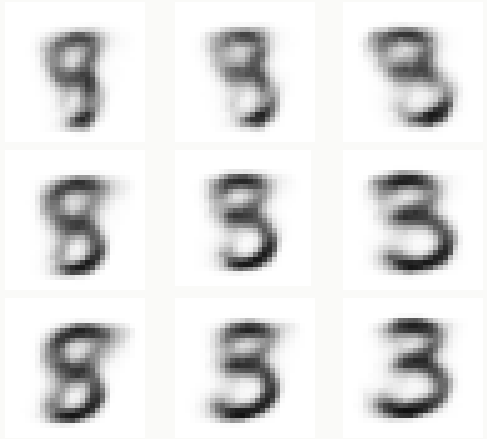
Selected z

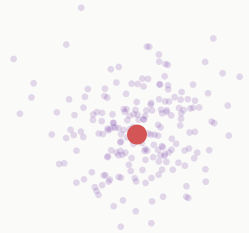


(0.00, 0.00)

Decoder probabilities

Local neighborhood

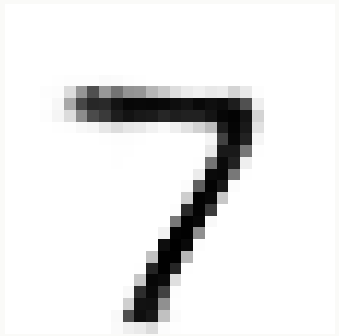




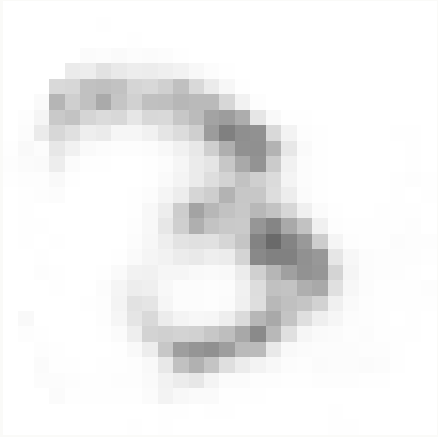
prior density: 0.159

MEASURED STATIC STATE · VAE-2 mean at z = (0, 0), with neighboring decodes.

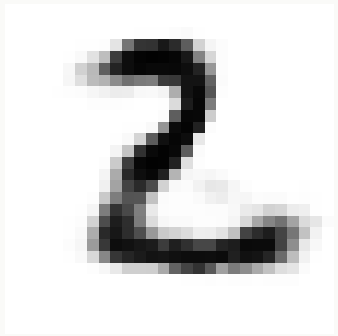
What changes between this 7 and this 2?



7 endpoint



t = 0.500



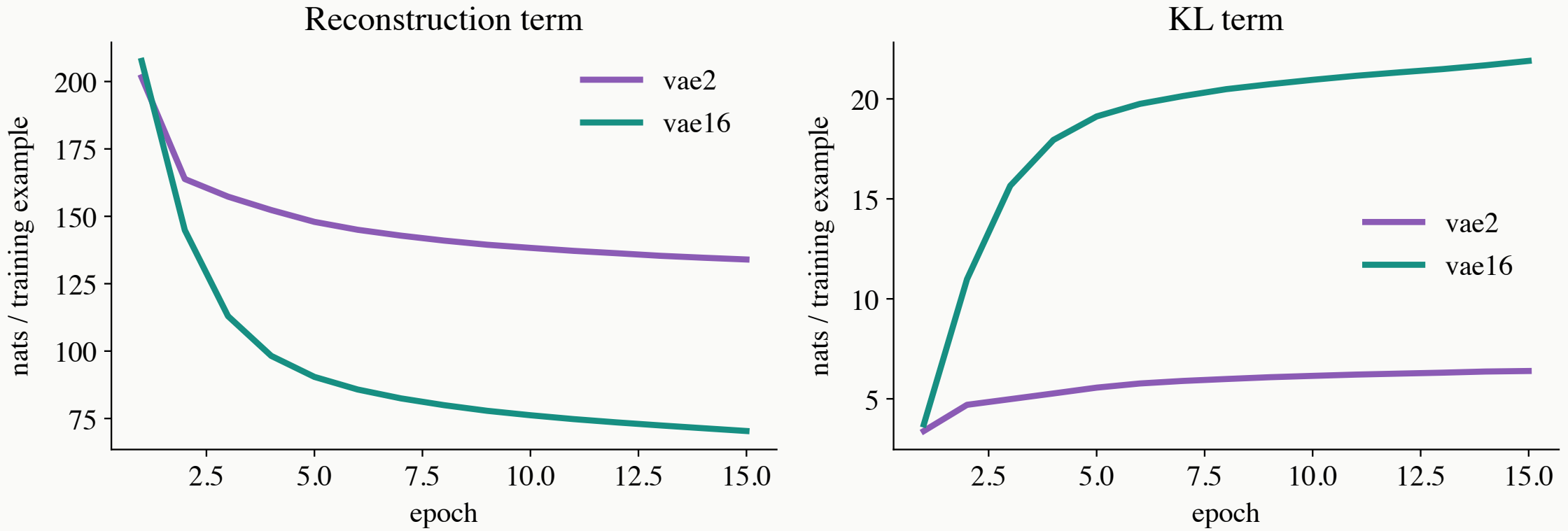
2 endpoint



$z(t) = (1 - t)\,\mu(x_7) + t\,\mu(x_2)$ in 16-dimensional latent space

41 saved decoder evaluations; the line shows path position t.

What changes when we train a larger latent representation?



Lower reconstruction loss alone does not establish better generation.

What changes when we strengthen the prior penalty?

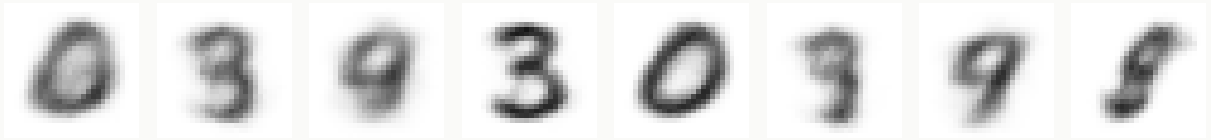
β = 4 · separately trained 2D VAE

Test recon 162.8 · KL 3.04 nats

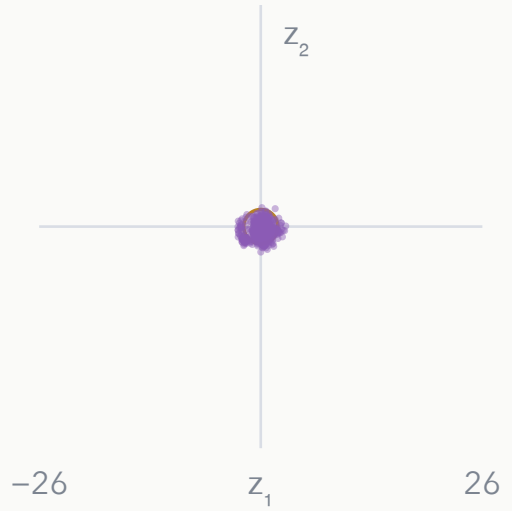
Held-out reconstructions



Same prior vectors → decoder means



600 posterior means



amber: prior radius 2

Same inputs, seeds, data order and training budget.

12k training images · 12 epochs · one seed. β = 0 still samples a stochastic latent.

Learning a distribution through classification

07

Can source classification teach a generator a distribution?

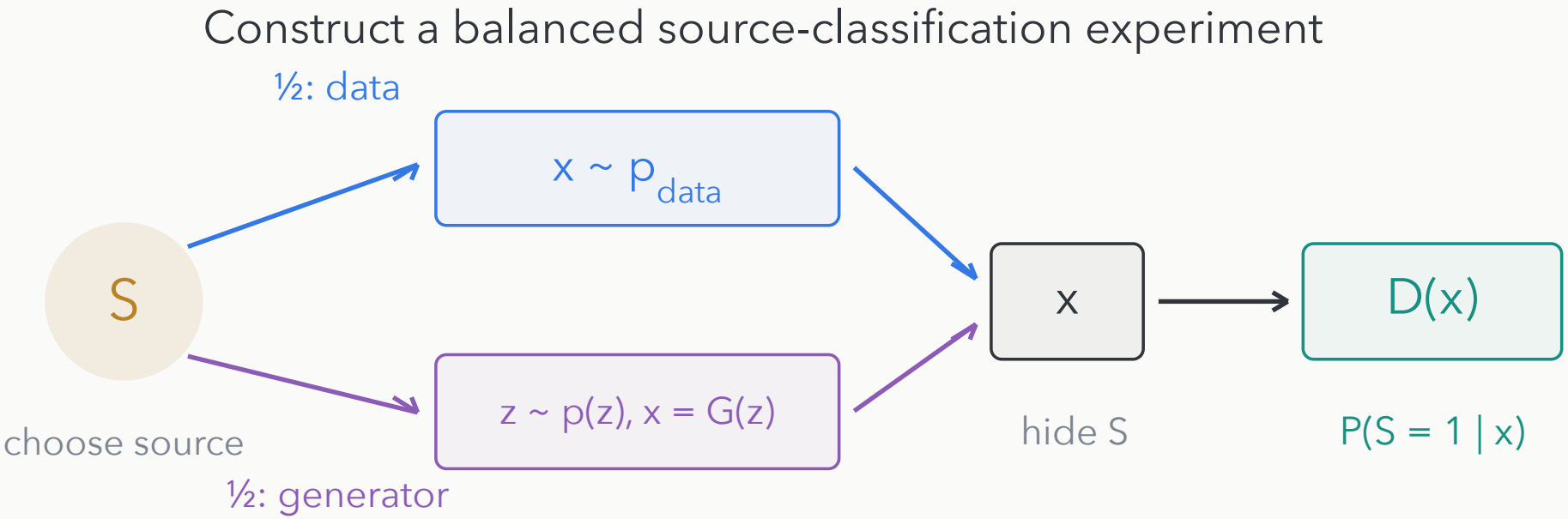
From randomness to data

Must we train with that particular objective?

Keep the generator. Learn a different training signal.



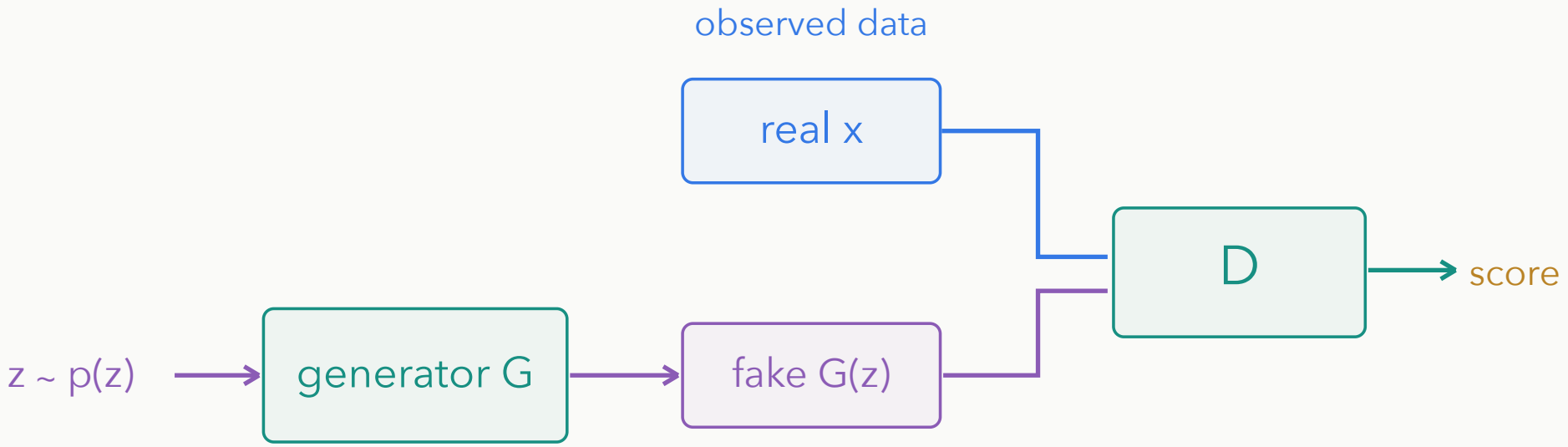
What probability does the discriminator predict?



The generator induces p_G over x through its map G(z).

D predicts the source in this experiment. Its output is not the data density.

How could generated points get feedback?



Train D to distinguish the two sources.

Where does the discriminator loss come from?

A Bernoulli model for the hidden source S

S = 1: probability D(x)

S = 0: probability 1 − D(x)

$$\log P(S \mid x) = S \log D(x) + (1 - S) \log(1 - D(x))$$

$$V(G,D) = E_{data}[\log D(x)] + E_{G}[\log(1 - D(x))]$$

Balanced average = V / 2. Train D by minimising $L_D = -V$.

Expectations use data samples and generated samples. No evaluation of p_G(x) is required.

What would the best discriminator say?

Data-source prior π = 0.50; generator-source prior = 0.50

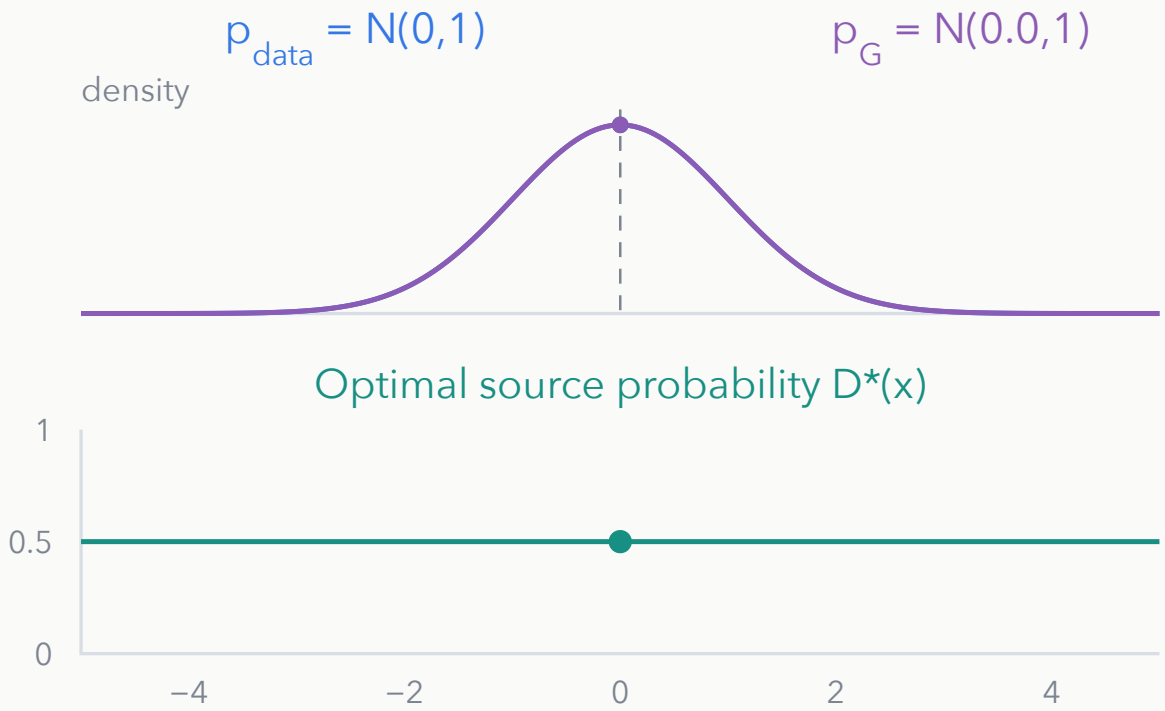
	A	B	C
$p_{data}(x)$	0.60	0.30	0.10
$p_G(x)$	0.10	0.30	0.60
$D^*(x)$	0.857	0.500	0.143

$$D^*(x) = \pi\, p_{data}(x) / [\pi\, p_{data}(x) + (1 - \pi)\, p_G(x)]$$

Equal priors: $D^* / (1 - D^*) = p_{data} / p_G$

Known toy probabilities; a neural discriminator learns the comparison from samples.

How does the discriminator change as the model moves?



Inspect x = 0.0

$p_{data}(x) = 0.399$

$p_G(x) = 0.399$

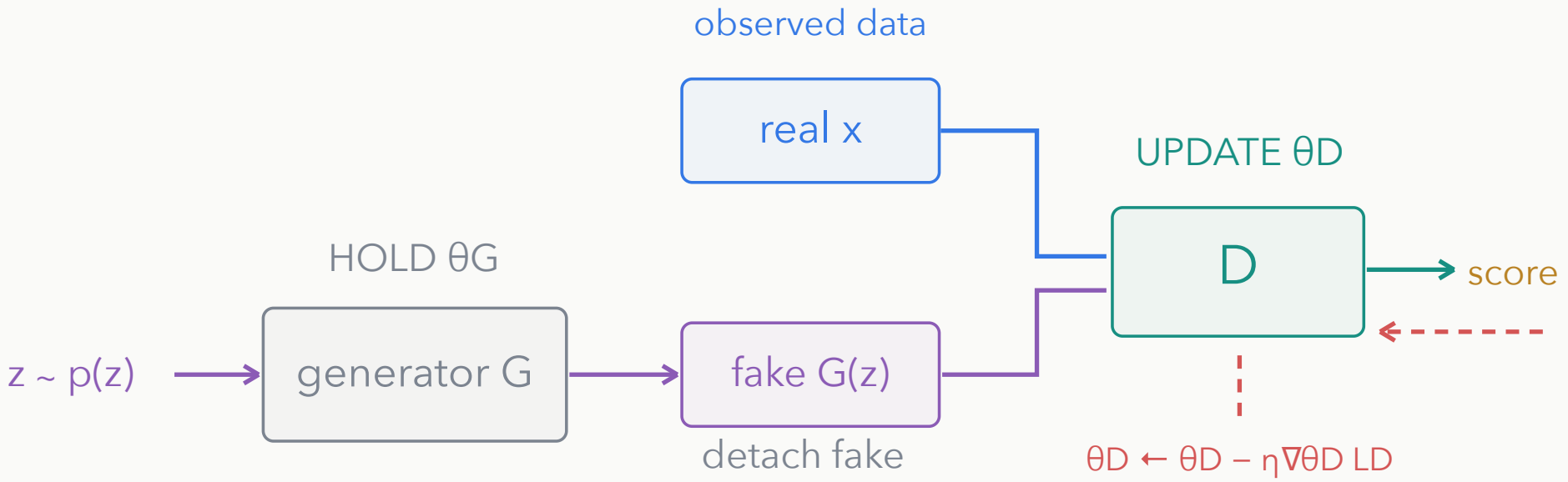
$D^*(x) = 0.500$

Same distributions

D* = 0.5 everywhere

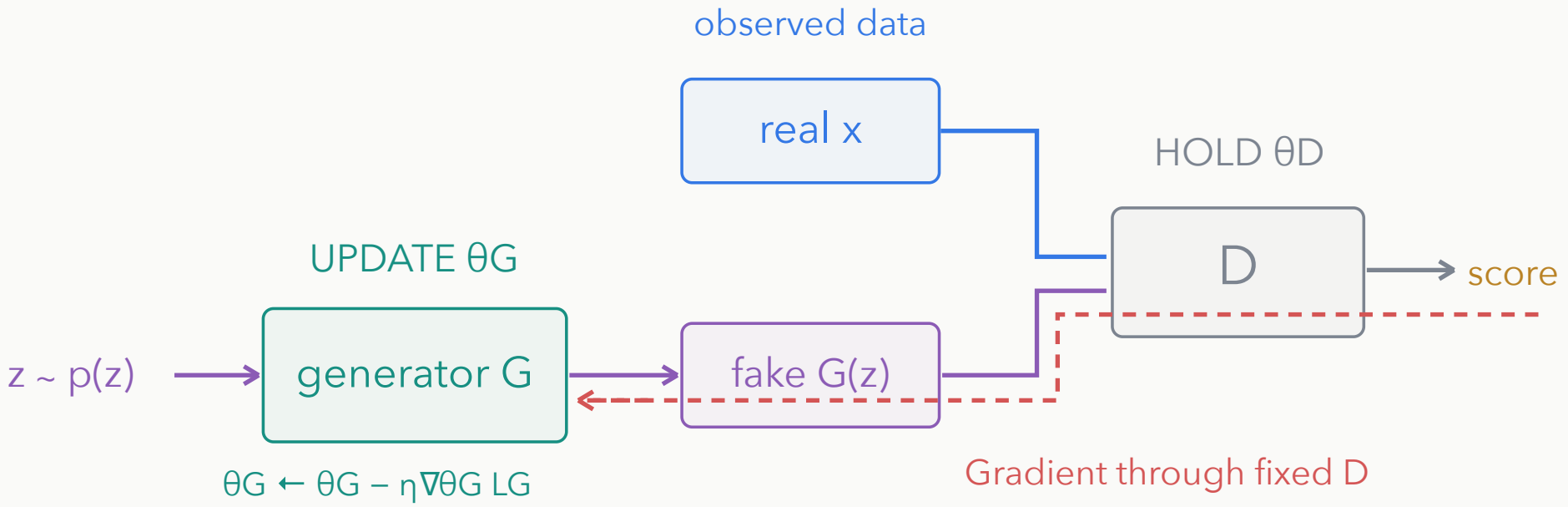
Analytical Gμ(z) = z + μ and optimal D*. This is not a recorded neural-training run.

What learns during the discriminator step?



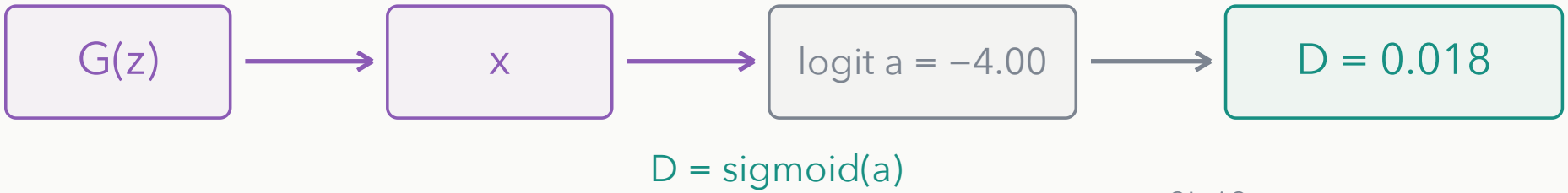
$$LD = -E \log D(x) - E \log(1 - D(G(z)))$$

Can fixed discriminator weights still pass gradients?



Why change the generator's loss in practice?

Hold D's weights fixed; keep its input-gradient path.

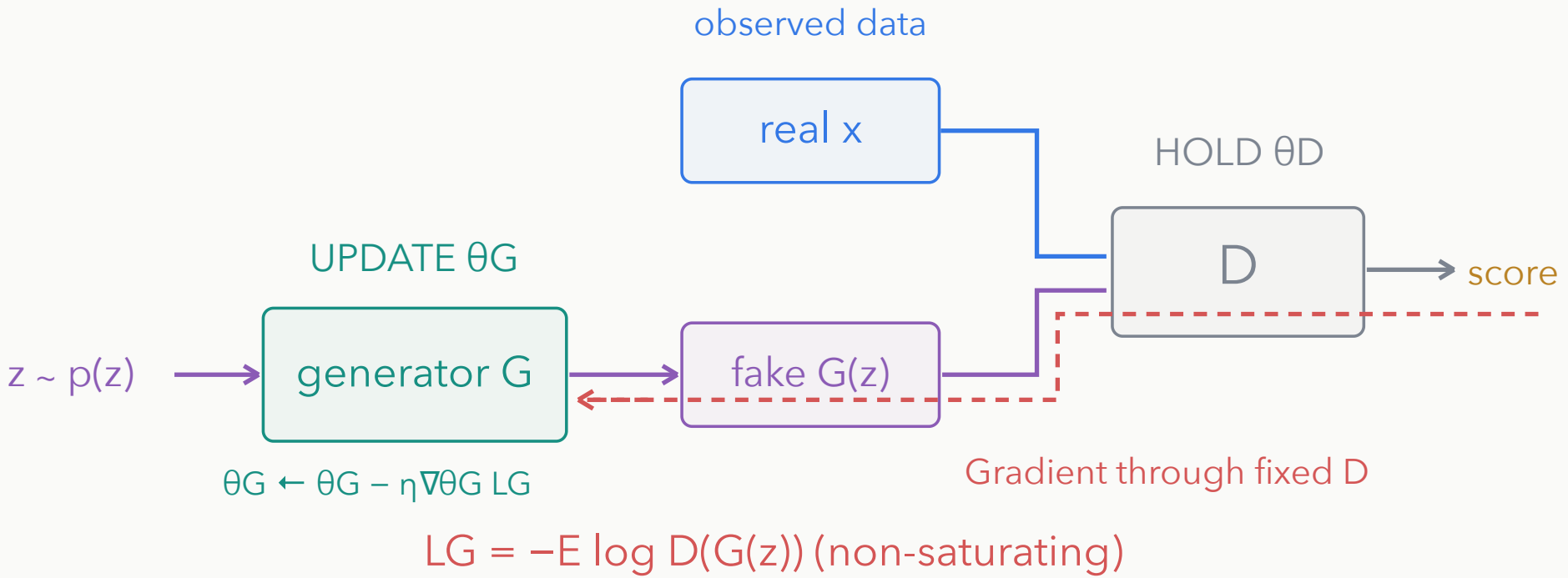


	∂L / ∂a	
minimax: log(1 − D)	−D	−0.018
non-saturating: −log D	D − 1	−0.982

These are ∂L/∂a. The full gradient also passes through D and G.

A larger loss-to-logit derivative does not guarantee a useful full generator gradient.

How do we train both networks?



G step: keep D weights fixed; keep its input-gradient path.

What do these two losses ask for?

$D(x) = 0.9$

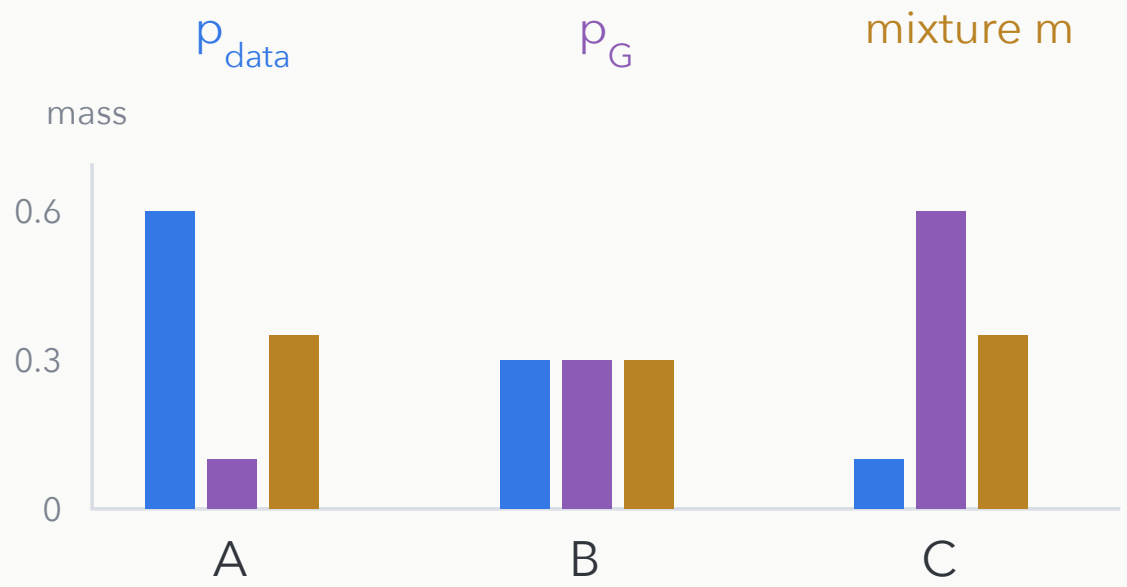
$D(G(z)) = 0.2$

$$LD = -\log 0.9 - \log(1 - 0.2) = 0.329$$

$$LG = -\log 0.2 = 1.609$$

Classical GAN source probability; a WGAN critic has a different interpretation.

How is the ideal game connected to KL?



$$m = (p_{data} + p_G) / 2$$

$$KL(p_{data} \| m) = 0.198$$

$$KL(p_G \| m) = 0.198$$

$$JS = ½ (0.198 + 0.198)$$

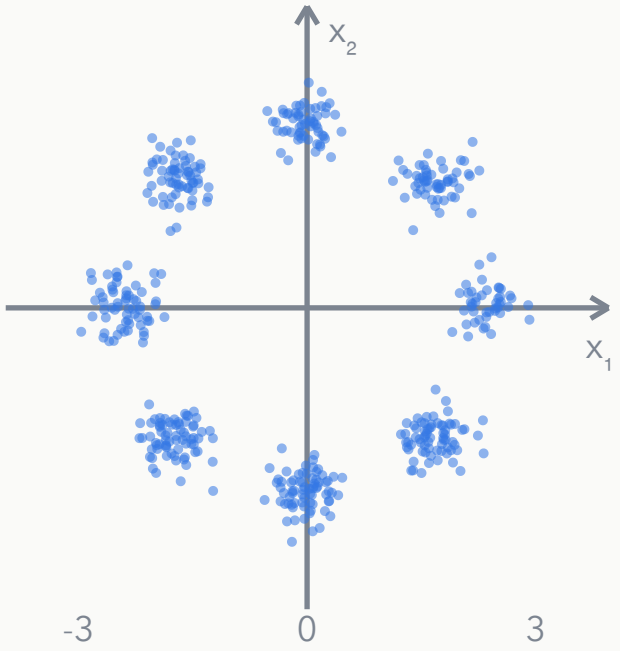
$$JS = 0.198 \text{ nats}$$

$$V(G,D^*) = -\log 4 + 2\,JS = -0.990$$

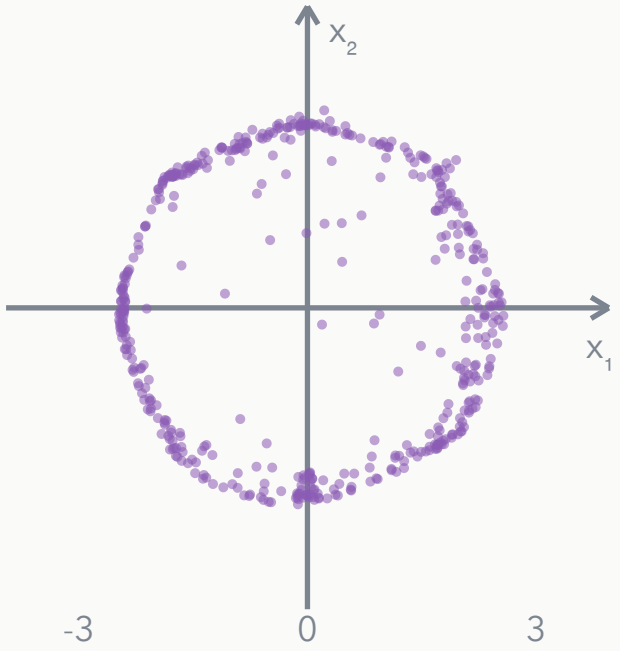
Optimal D and the classical minimax objective. Practical non-saturating updates differ.

What does alternating training do to the points?

Real mixture samples



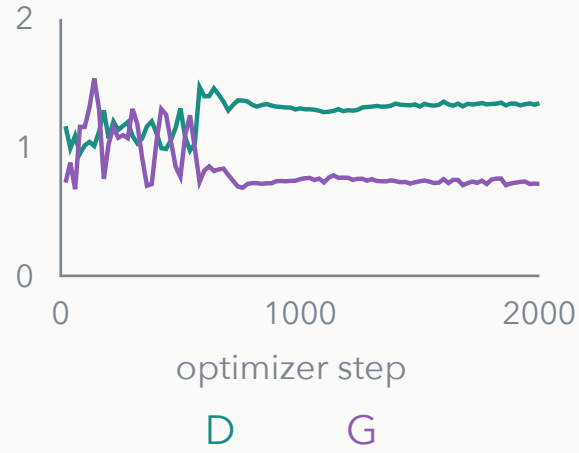
Generated · step 2000



8 / 8 modes covered

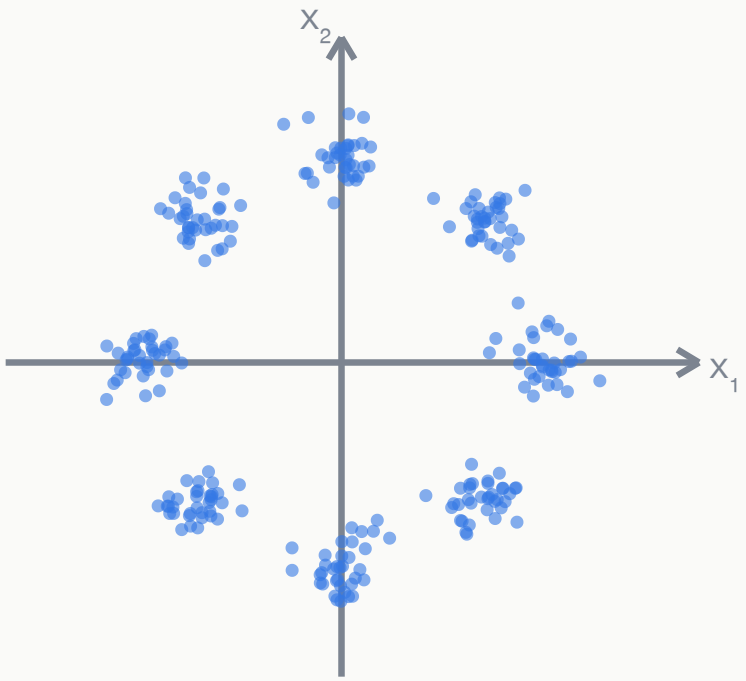
77.6% near a mode

Minibatch loss (nats)

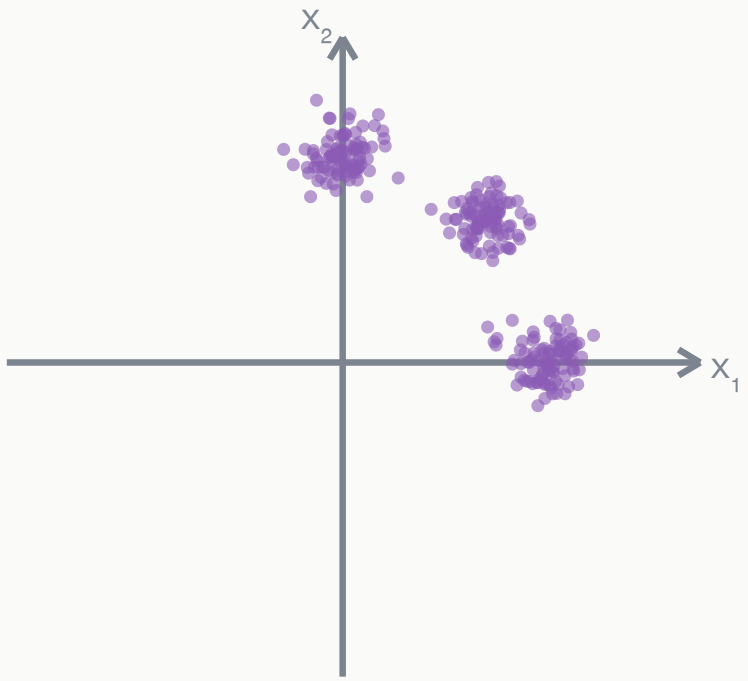


Coverage: ≥1% of 1,000 fixed samples within 3σ of a centre. Losses are recorded minibatch values.

If samples look plausible, have we succeeded?



Data: eight modes



Illustrative generator: 3 modes

Constructed coverage: 3 / 8 modes · this is not a GAN training run.

> Sample quality and distribution coverage are different questions.

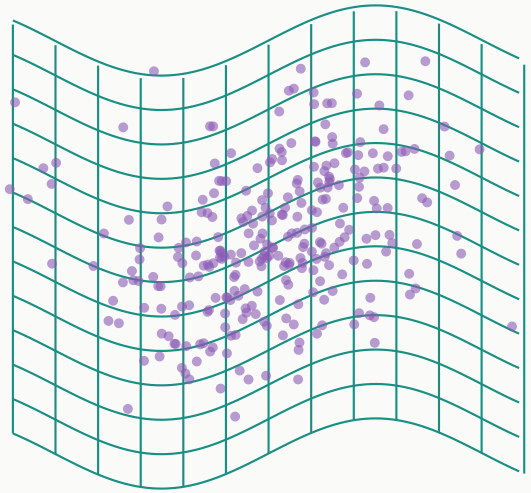
Invertible generation

08

Could many small transformations make generation easier?

From randomness to data

What if our generator were invertible?



A smooth shear

No folding. No cutting.

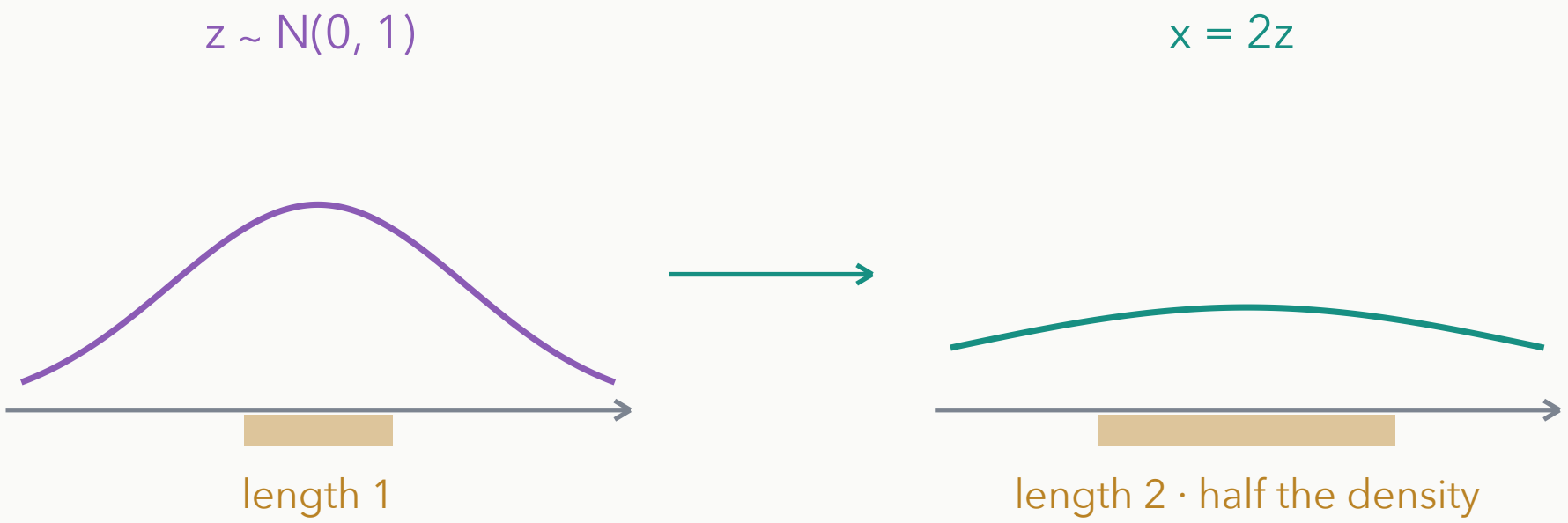
Same points and grid lines

Illustrative map; not a fitted model

Invertible shear; determinant 1. This illustrative map preserves local area.

$$z = f^{-1}(x)$$

Where does exact density come from?



$$\log p_X(x) = \log p_Z(f^{-1}(x)) + \log \left|\det J f^{-1}(x)\right|$$

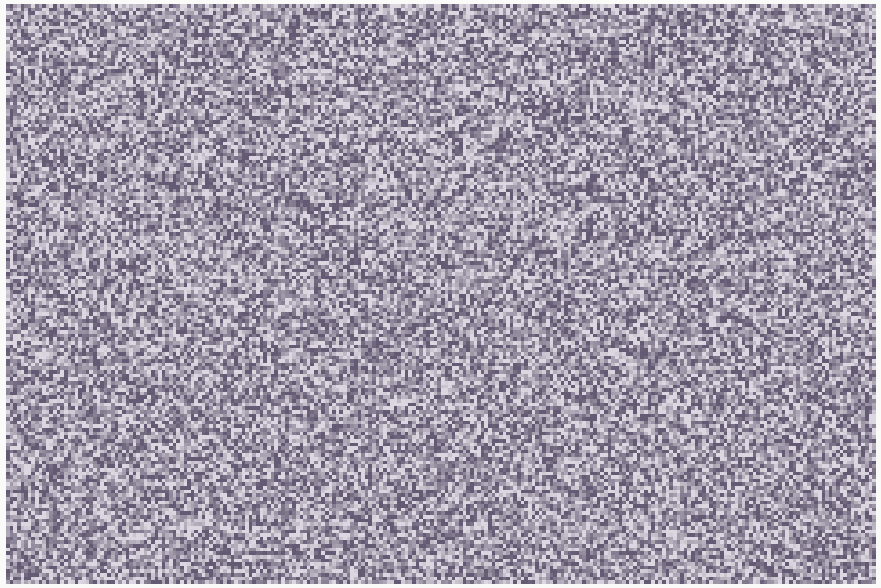
Both highlighted intervals contain probability 0.383. Inverse Jacobian = 0.5.

Four answers to the same question

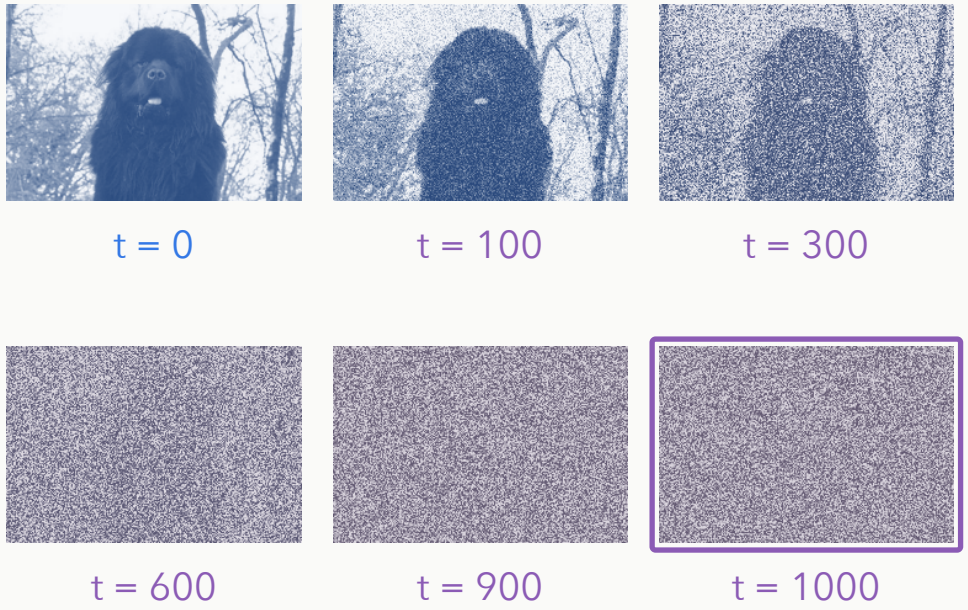
Autoregressive	Sample one part conditioned on earlier parts.
VAE	Sample a latent prior; decode with a probability model.
GAN	Learn a generator through adversarial feedback.
Flow	Invertibly transform a known density.

> How do we learn to turn randomness into valid data?

Could many small transformations help?



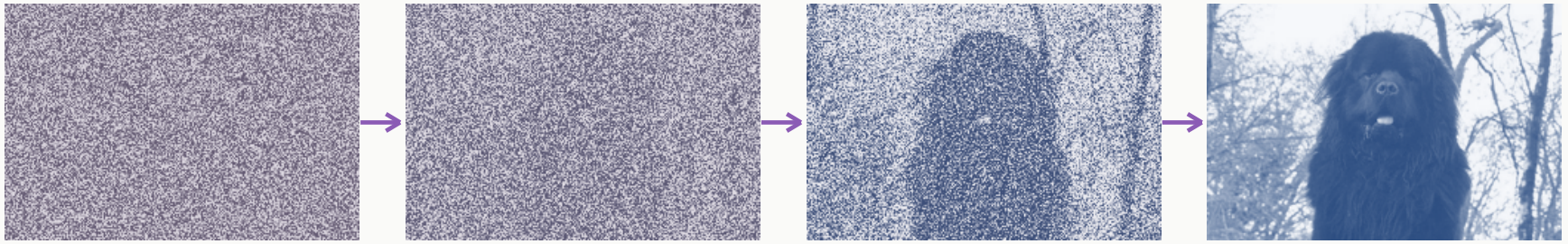
Saved forward step 1000



Same six frames used in Generative Modelling II.

> This direction is easy.

Can we learn to remove a little noise at a time?



noise → learned reverse steps → data?

Could a learned model run this corruption process backwards?